

Technische Hochschule Nürnberg Georg Simon Ohm
Fakultät Elektrotechnik Feinwerktechnik Informationstechnik

Implementierung eines fiducialbasierten Lokalisierungssystems für Luftfahrzeuge im Innenraum

Masterarbeit im Studiengang Elektronische und Mechatronische Systeme
Vertiefungsrichtung Automatisierungstechnik

Hannes Müller
3796115

Betreuer: Prof. Dr. rer. nat. Pfitzner
Betreuer: Noch nicht sicher

Abgabe: Nürnberg, 10. Juni 2026
Sommersemester 2026

Abstract

This thesis describes the development of a visual localization system for unmanned aerial vehicles based on fiducial markers in indoor environments. The Robot Operating System 2 is used as the middleware framework; AprilTags are used as the marker framework. The system uses the RGB images from two cameras as its input data source—one facing forward and one facing downward. The image processing and navigation calculations are performed by a mini-computer mounted on the aircraft. Position determination is based on the mapping of fiducial markers using simultaneous localization and mapping. For the extrinsic calibration of the cameras, a custom-developed algorithm for determining the respective transformation matrix is described. Position determination itself is achieved by detecting the previously mapped markers and calculating the poses of the camera lenses. The position data from both sensors are processed, fused into a single pose, and transmitted to the flight controller. The overall system enables GPS-independent positioning designed for indoor use. Finally, a prototype application is presented that can be used to control the aircraft.

Kurzfassung

Diese Arbeit beschreibt die Entwicklung eines visuellen Lokalisierungssystems für unbemannte Luftfahrzeuge auf Basis von Fiducial-Markern im Innenraum. Als Middleware-Framework wird das Robot Operating System 2 verwendet; als Marker-Framework kommen AprilTags zum Einsatz. Das System nutzt die RGB-Bilder zweier Kameras als Eingangsdatenquelle – eine frontal und eine nach unten ausgerichtet. Die Berechnungen zur Bildverarbeitung sowie zur Navigation werden durch einen am Luftfahrzeug montierten mini-Computer durchgeführt. Grundlage der Positionsbestimmung bildet die Kartierung der Fiducial-Marker mittels Simultaner Lokalisierung und Kartierung. Zur extrinsischen Kalibrierung der Kameras wird ein eigens entwickelter Algorithmus zur Bestimmung der jeweiligen Transformationsmatrix beschrieben. Die Positionsbestimmung selbst erfolgt durch die Erkennung der zuvor kartierten Marker sowie die Berechnung der Posen der Kameraoptiken. Die Positionsdaten beider Sensoren werden aufbereitet, zu einer einzigen Pose fusioniert und an den Flugcontroller übermittelt. Das Gesamtsystem ermöglicht eine GPS-unabhängige Lokalisierung, die für den Einsatz in Innenräumen ausgelegt ist. Abschließend wird eine prototypische Anwendung vorgestellt, die zur Steuerung des Luftfahrzeugs eingesetzt werden kann.

Inhaltsverzeichnis

Abstract	III
Kurzfassung	IV
Abkürzungsverzeichnis	VIII
1 Einleitung	1
2 Theoretische Grundlagen	2
2.1 Grundlagen Unbemannter Flugroboter	2
2.1.1 Mechanischer Aufbau am Beispiel der Holybro X650	2
2.1.2 Vergleich von Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem	2
2.2 Repräsentation von Position und Orientierung	4
2.2.1 Quaternionen	4
2.2.2 Rodrigues-Parameter	6
2.3 Transformation von Koordinatensystemen	6
2.4 Sensorsysteme	7
2.5 Extended Kallman Filter	8
2.6 Flugcontroller Pixhawk 6X	9
2.6.1 Sensorik	10
2.6.2 Schnittstellen	10
2.6.3 EKF2 – Zustandsschätzung und Sensorfusion	11
2.6.4 Betriebssystem	11
2.6.5 Random Sample Consensus (RANSAC)	11
2.6.6 Robot Operating System 2 (ROS2)	12
2.6.7 Nodes und Topics	12
2.6.8 Bag-Dateien	12
2.6.9 TF2 – Koordinatentransformationen	12
3 Stand der Technik	14
3.1 GNSS-freie Lokalisierung von UAVs	14
3.1.1 Absolute Lokalisierung	14
3.1.2 Relative Lokalisierung	15
3.1.2.1 Dead Reckoning Localization	15
3.1.2.2 Visuelle Odometrie und optischer Fluss	15
3.1.3 Simultane Lokalisierung und Kartierung	16
3.1.3.1 Phasen optimierungsbasierter SLAM Methoden	16
3.1.3.2 Faktorgraph Optimierung	17
3.1.3.3 SLAM-Varianten	18
3.2 Fiducial-Marker-Systeme	19
3.2.1 ARTag Marker	19

3.2.2	AprilTag Marker	20
3.2.3	ArUco Marker	20
3.2.4	STag Marker	20
3.2.5	Detektion von Fiducial-Markern	20
4	Systembeschreibung	22
4.1	Anforderungen an das Gesamtsystem	22
4.2	Hardwareaufbau	22
4.3	Softwarearchitektur Kartierung und Positionierung nochmal drüber schauen!!! ich nehme ja auch das thinkpad!!!	25
4.4	Versuchsumgebung	26
5	Implementierung der Kartierung	28
5.1	Auswahl der Algorithmik	28
5.1.1	Auswahl der Bewertungskriterien SLAM-Varianten	28
5.1.2	Gewichtung durch den paarweisen Vergleich der SLAM-Varianten	28
5.1.3	Nutzwertanalyse SLAM-Varianten	29
5.1.4	Auswahl der Fiducial-Marker	31
5.2	Ablauf der Kartierung	32
5.2.1	Kamera-Kalibrierung	32
5.2.2	Bildaufnahme Kartierung	35
5.2.3	Entzerrung	35
5.2.4	Offline Detektion von AprilTags	36
5.2.5	Kartenerstellung und Konfiguration von Taglam	37
6	Implementierung der Positionierung	39
6.1	Softwarearchitektur der Positionierung	39
6.2	Bildaufnahme Positionierung	39
6.2.1	Kamerakalibrierung Positionierung	40
6.2.2	Kamerakalibrierung Positionierung	40
6.3	ab hier ist vorsicht geboten	41
6.4	Entzerrung	41
6.5	Bildverarbeitung	41
6.6	Positionsbestimmung	41
6.6.1	Posenberechnung je Marker	41
6.6.2	Ausreißerfilterung	42
6.6.3	Gewichtete Fusion	42
6.6.4	Umrechnung auf den Drohnenmittelpunkt	42
6.6.5	Kovarianzschätzung	42
6.6.6	Extrinsische Kamerakalibrierung	43

6.7	Kommunikation mit der Flugsteuerung	43
6.7.1	Koordinatentransformation	43
6.7.2	Zeitbezug und Ausgabe	43
6.8	Flugsteuerung	44
7	Systemintegration	45
8	Evaluation des Greifens	46
8.1	Erfolgskriterien - Erfolgsquote und Fehlerrate	46
8.2	Erkennungsgenauigkeit	46
8.3	Positioniergenauigkeit	46
8.4	Zeitliche Performance und Echtzeitfähigkeit	46
9	Zusammenfassung und Ausblick	47
10	Anhang	56

Abkürzungsverzeichnis

DRL	Dead Reckoning Localization
EKF	Extended Kalman Filter
EKF2	Extended Kalman Filter 2
EMA	Exponential Moving Average
FMU	Flight Management Unit
fps	frames per second
g2o	General Graph Optimization
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GPU	Graphics Processing Unit
GTSAM	Georgia Tech Smoothing and Mapping
IEKF	Invariant Extended Kalman Filter
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
LiPo	Lithium-Polymer-Akku
OIC	Ohm Innovation Center
PX4	Pixhawk
RANSAC	Random Sample Consensus
ROS2	Robot Operating System 2
SLAM	Simultaneous Localization and Mapping
TOPS	Tera Operations Per Second
UAV	Unmanned Aerial Vehicle
UDP	User Datagram Protocol
USB	Universal Serial Bus
VIO	Visuelle Inertial Odometrie
VO	Visuelle Odometrie
vSLAM	Visual SLAM

1 Einleitung

Die vorliegende Arbeit beschäftigt sich mit der Entwicklung eines Fiducial-basierten Systems zur Positionserkennung eines Flugroboters im Innenraum. Als Grundgerüst des Systems dient eine Holybro X650 Unmanned Aerial Vehicle (UAV), welche mit einem Pixhawk 6X Flight Controller ausgestattet ist. Es wird auf die Parametrierung und Kalibrierung des Flight Controllers eingegangen, sodass die vom implementierten System bereitgestellte Position mit den übrigen Sensordaten durch einen Extended Kalman Filter fusioniert werden kann. Der Flight Controller ist seriell mit einem Jetson Orin NX verbunden, auf welchem ein Robot Operating System 2 (ROS2) Humble System läuft, das die Implementierung der Algorithmen sowie die Kommunikation zwischen den Komponenten ermöglicht. Die Kommunikation zwischen dem Jetson Orin NX und dem Pixhawk 6X wird über das MAVROS-Paket realisiert.

Als Sensoren zur Erkennung der Fiducials werden zwei Logitech-Kameras am Flugroboter angebracht und mit dem Jetson Orin NX verbunden, wobei eine Kamera nach unten und eine nach vorne ausgerichtet ist. Zur Auswahl der geeigneten Kameras werden die Logitech C920 Pro und die Logitech C922 Pro anhand definierter Bewertungskriterien verglichen, die mithilfe eines Paarweisen Vergleichs gewichtet und anschließend in einer Nutzwertanalyse bewertet werden. Aufgrund des Ergebnisses wird die Logitech C920 als nach vorne gerichtete Kamera und die Logitech C922 als nach unten gerichtete Kamera ausgewählt. Die Kameras werden mittels extrinsischer Kalibrierung geometrisch aufeinander abgestimmt.

Um eine zuverlässige Positionserkennung zu gewährleisten, werden AprilTags, ArUco-Marker und WhyCon-Marker ebenfalls anhand einer gewichteten Nutzwertanalyse verglichen. AprilTags gehen als geeignetstes Fiducial hervor. Für die Kartierung des Innenraums wird die Erkennung der AprilTags über die OpenCV-Bibliothek implementiert, während die Live-Positionierung durch die hardwarebeschleunigte NVIDIA Isaac ROS2 AprilTag-Erkennung auf dem Jetson Orin NX realisiert wird.

Die Kartierung des Innenraums erfolgt auf Basis einer Faktorgrafen-basierten Zustandsschätzung. Ein erster Ansatz der Echtzeit-Kartierung wird aufgrund der zu hohen Rechenlast verworfen. In einem zweiten Ansatz wird die Kartierung offline auf einem separaten Computer unter ROS2 Jazzy durchgeführt. Die erstellten Karten werden anhand der realen Umgebung bewertet und mit verschiedenen Parametern optimiert.

Die Positionserkennung relativ zur erstellten Karte wird ebenfalls in zwei Varianten untersucht. Zum einen die Faktorgrafen-basierte Zustandsschätzung, zum anderen die direkte geometrische Posenschätzung. Aufgrund der geringeren Rechenlast und der verfügbaren hardwarebeschleunigten NVIDIA Isaac AprilTag-Unterstützung wird die direkte geometrische Posenschätzung eingesetzt. Die erkannten Tags werden mithilfe von Exponential Moving Average (EMA)-Glättung, Medianfilterung, Random Sample Consensus (RANSAC) und Kovarianzschätzung aufbereitet und dem Flight Controller bereitgestellt.

Abschließend werden Probleme und Schwachstellen des Systems diskutiert sowie Verbesserungspotenziale aufgezeigt.

2 Theoretische Grundlagen

Zum besseren Verständnis der vorliegenden Arbeit werden im folgenden Kapitel die theoretischen Grundlagen erläutert. Es werden grundlegende Begriffe aus den Bereichen der Robotik, der Sensorik sowie der visuellen Navigation eingeführt. Darüber hinaus wird gesondert auf den verwendeten Flugcontroller, die zugrundeliegenden Koordinatensysteme sowie die relevanten Transformationen eingegangen.

2.1 Grundlagen Unbemannter Flugroboter

Unter einem unbemannten Luftfahrzeug UAV, umgangssprachlich auch als Drohne bezeichnet, versteht man ein Fluggerät, welches ohne einen menschlichen Piloten an Bord betrieben werden kann. Dabei können UAVs entweder ferngesteuert von einem menschlichen Operator oder vollständig autonom durch einen bordeigenen Computer geführt werden. Die Anfänge unbemannter Luftfahrzeuge reichen bis in die 1910er Jahre zurück, wo sie vorrangig für militärische Zwecke eingesetzt wurden. Ab den 2000er Jahren fanden UAVs zunehmend Eingang in zivile Anwendungsgebiete, wie beispielsweise die Katastrophenhilfe [1, S. 1–3].

2.1.1 Mechanischer Aufbau am Beispiel der Holybro X650

Grundrahmen: Der Grundrahmen eines UAV dient als mechanische Basis für die Montage aller weiteren Komponenten. Das Design des Rahmens beeinflusst maßgeblich die Flugeigenschaften des Geräts. Das in dieser Arbeit verwendete Modell ist als Senkrechtstarter konzipiert und basiert auf einem X-förmigen Quadcopter-Rahmen. Die wesentlichen Bestandteile des Rahmens umfassen die Ausleger, das Landegestell sowie die zentrale Montageplattform für die Elektronik. Die Holybro X650 weist eine Diagonale von 650 mm auf. Abbildung 2.2 zeigt ein Bild der Holybro X650. [2] [3]

Antriebssystem: Das Antriebssystem besteht aus vier Elektromotoren, die jeweils an einem der Ausleger montiert sind. An den Motoren sind Propeller befestigt, welche den erforderlichen Auftrieb erzeugen.

Flugcontroller: Der Flugcontroller bildet das zentrale Steuerungselement eines UAV. Er setzt eingehende Steuerbefehle in konkrete Ansteuersignale um, die an die Motoren weitergeleitet werden. In Kapitel ?? wird näher auf den verwendeten Flugcontroller eingegangen.

Energieversorgung: Die Energieversorgung erfolgt über einen Lithium-Polymer-Akku (LiPo). Dieser versorgt die Steuerelektronik als auch die Motoren.

Nutzlast: Das maximale Abfluggewicht der Holybro X650 beträgt 6,3 kg, wobei der flugfertige Grundrahmen mit Antriebssystem bereits ein Eigengewicht von 4,3 kg aufweist. Daraus ergibt sich eine maximale Nutzlast von 2 kg (ohne Akku) [3].

2.1.2 Vergleich von Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem

Dieses Kapitel behandelt die verschiedenen Koordinatensysteme, die in dieser Arbeit relevant sind. Dabei wird zwischen dem Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem unterschieden.

Weiterhin werden die grundlegenden Konzepte zur Repräsentation von Position und Orientierung im Raum erläutert.

Ein Koordinatensystem ist ein mathematisches Konstrukt, das zur eindeutigen Beschreibung der Lage von Punkten im Raum verwendet wird. Zur Beschreibung von Bewegungen ist die Festlegung eines Koordinatensystems von entscheidender Bedeutung. Es besteht aus einem Bezugspunkt, dem Ursprung, sowie drei orthogonalen Achsen, die als X-, Y- und Z-Achse bezeichnet werden. In dieser Arbeit wird standardmäßig das rechtshändige Koordinatensystem verwendet [4, S. 17]. Mathematisch wird der Zusammenhang durch

$$\mathbf{Z} = \mathbf{X} \times \mathbf{Y} \quad (2.1)$$

dargestellt. In dieser Arbeit werden die Achsen entsprechend des informellen Industriestandards wie folgt dargestellt: X-Achse in Rot, Y-Achse in Grün und Z-Achse in Blau. Ein Überblick über die in dieser Arbeit verwendeten Koordinatensysteme ist in Abbildung 2.1 dargestellt.

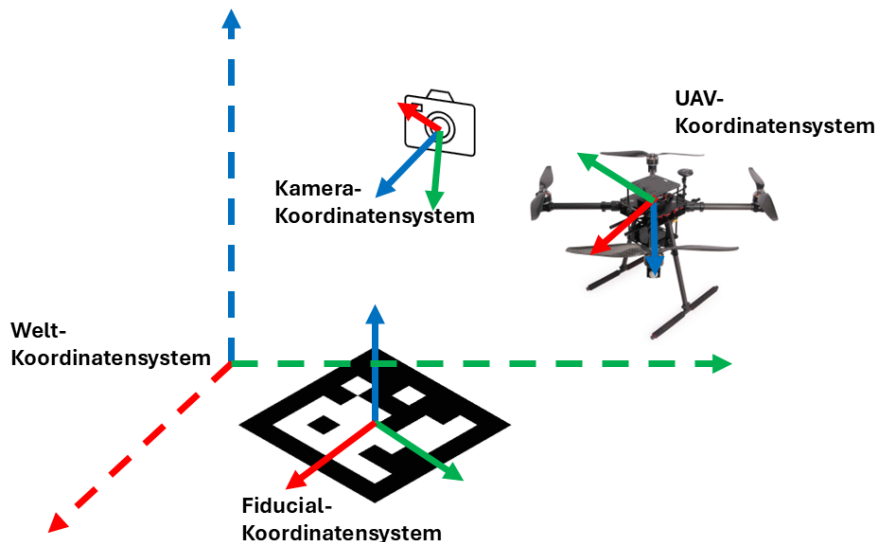


Abbildung 2.1: Vergleich der Koordinatensysteme [3]

Das Weltkoordinatensystem dient als globaler Referenzrahmen, in dem die Position aller Objekte beschrieben wird. Der Ursprung dieses Koordinatensystems kann frei gewählt werden und wird durch die Variable O_0 repräsentiert. Die Achsen werden durch X_0 , Y_0 und Z_0 dargestellt [5, S. 4].

Das UAV-Koordinatensystem ist in der Regel vom Hersteller vorgegeben und richtet sich in diesem konkreten Fall nach dem Flugregler. In der zugehörigen Dokumentation wird angegeben, dass die X-Achse (X_u) nach vorne, die Y-Achse (Y_u) nach rechts und die Z-Achse (Z_u) nach unten zeigt. Der Ursprung dieses Koordinatensystems (O_u) befindet sich im Zentrum des Flugreglers [6].

Das Kamerakoordinatensystem ist in der Regel so definiert, dass die Z-Achse (Z_c) entlang der optischen Achse der Kamera zeigt, die X-Achse (X_c) nach rechts auf der Bildebene und die Y-Achse (Y_c) nach unten auf der Bildebene zeigt. Der Ursprung (O_c) befindet sich im optischen Zentrum der Kamera [7, S. 44].

Das Fiducialkoordinatensystem ist in dieser Arbeit so definiert, dass sich der Ursprung (O_f) im Mittelpunkt der Fiducialmarke befindet. Die Z-Achse (Z_f) zeigt senkrecht aus der Markerebene heraus, die X-Achse (X_f) zeigt nach rechts in der Markerebene und die Y-Achse (Y_f) zeigt nach oben in der Markerebene.

2.2 Repräsentation von Position und Orientierung

In diesem Unterkapitel wird das verwendete UAV als starrer Körper betrachtet. Die vollständige Beschreibung der Lage eines starren Körpers im Raum erfolgt durch seine Position und Orientierung [8, S. 12].

Die Position eines Körpers wird durch den Vektor

$$\mathbf{p} = (x, y, z)^T \quad (2.2)$$

beschrieben, wobei die Koordinaten einem bestimmten Koordinatensystem zugeordnet sind.

Die Orientierung eines Körpers kann auf verschiedene Weisen dargestellt werden. Im Folgenden werden die zum Verständnis dieser Arbeit relevanten Darstellungsformen erläutert. Grundsätzlich kann sich ein Körper um drei Achsen drehen, was durch die Euler-Winkel Roll (ϕ), Pitch (θ) und Yaw (ψ) beschrieben wird. Abbildung 2.2 zeigt die Definition dieser Winkel am Beispiel der Holybro X650. Die Pfeilrichtung entspricht der positiven Drehrichtung im rechtshändigen Koordinatensystem.

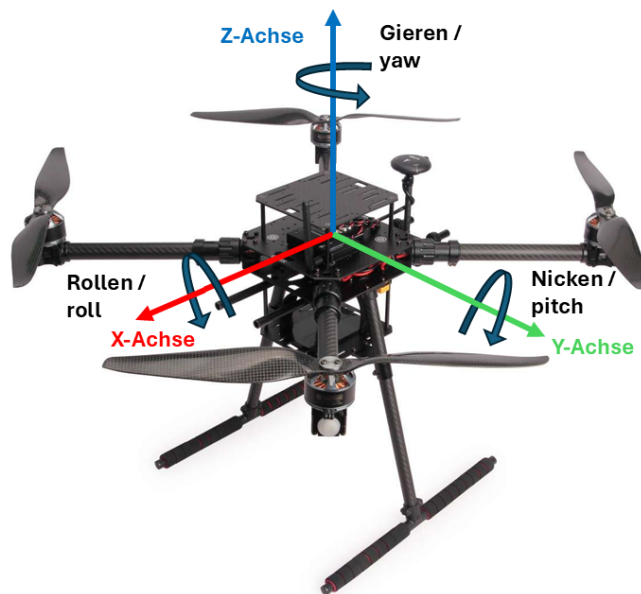


Abbildung 2.2: Roll, Pitch und Yaw am Beispiel Holybro X650 [3]

2.2.1 Quaternionen

Die Orientierung im Raum kann ebenfalls als Quaternion dargestellt werden. Quaternionen bieten gegenüber Euler-Winkeln den Vorteil, dass sie keine Singularitäten (Gimbal Lock) aufweisen. Qua-

ternionen stellen eine Erweiterung des komplexen Zahlenraums dar und werden durch Gleichung 2.3 definiert [8].

$$\mathbf{q} = w + ix + jy + kz \quad (2.3)$$

wobei die Komponenten w, x, y und z reelle Skalare sind und i, j sowie k die imaginären Einheiten darstellen. Die imaginären Einheiten erfüllen die folgenden Multiplikationsregeln:

$$ii = jj = kk = ijk = -1 \quad (2.4)$$

$$ij = k, \quad jk = i, \quad ki = j \quad (2.5)$$

$$ji = -k, \quad kj = -i, \quad ik = -j \quad (2.6)$$

Addition und Multiplikation von Quaternionen sind durch die Gleichungen 2.7 und 2.8 definiert.

$$q_1 + q_2 = (w_1 + w_2) + i(x_1 + x_2) + j(y_1 + y_2) + k(z_1 + z_2) \quad (2.7)$$

$$\begin{aligned} q_1 \cdot q_2 = & (w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2) \\ & + i(w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2) \\ & + j(w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2) \\ & + k(w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2) \end{aligned} \quad (2.8)$$

Das Einheitsquaternion $\mathbf{q}$ beschreibt eine Rotation um den Winkel ϕ um eine normierte Rotationsachse $\mathbf{v} = (v_x, v_y, v_z)^T$ und wird durch Gleichung 2.9 beschrieben [9].

$$\mathbf{q} = \left(\cos \frac{\phi}{2}, \mathbf{v} \cdot \sin \frac{\phi}{2} \right) = \left(\cos \frac{\phi}{2}, v_x \sin \frac{\phi}{2}, v_y \sin \frac{\phi}{2}, v_z \sin \frac{\phi}{2} \right) \quad (2.9)$$

Die Konjugation eines Quaternions ist analog zu den komplexen Zahlen durch Gleichung 2.10 definiert.

$$\bar{\mathbf{q}} = (w, -x, -y, -z) \quad (2.10)$$

Für die Rotation eines beliebigen Punktes $\mathbf{p} = (0, p_x, p_y, p_z)$ mittels eines Rotationsquaternions $\mathbf{q}$ wird Gleichung 2.11 angewendet. [10, S. 112]

$$\mathbf{p}' = \mathbf{q} \cdot \mathbf{p} \cdot \bar{\mathbf{q}} \quad (2.11)$$

Die Normalisierung eines Quaternions erfolgt gemäß

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}, \quad \|\mathbf{q}\| = \sqrt{x^2 + y^2 + z^2 + w^2} \quad (2.12)$$

2.2.2 Rodrigues-Parameter

Ein Rodrigues-Vektor ist eine alternative Möglichkeit, eine Rotation im Raum zu beschreiben. Die Rotation wird durch einen Vektor $\mathbf{r} = (r_x, r_y, r_z)^T$ beschrieben, dessen Richtung die Rotationsachse angibt und dessen Betrag den Rotationswinkel repräsentiert [11]. Der Rotationswinkel ϕ ergibt sich aus dem Betrag des Rodrigues-Vektors gemäß Gleichung 2.13 und wird gegen den Uhrzeigersinn um die Rotationsachse gemessen. [12]

$$\phi = \|\mathbf{r}\| = \sqrt{r_x^2 + r_y^2 + r_z^2} \quad (2.13)$$

Für die Umrechnung in ein Quaternion muss die Rotationsachse $\mathbf{v}$ zunächst normiert werden:

$$\hat{\mathbf{v}} = \frac{\mathbf{r}}{\phi} = \left(\frac{r_x}{\phi}, \frac{r_y}{\phi}, \frac{r_z}{\phi} \right)^T \quad (2.14)$$

Die Umrechnung in ein Einheitsquaternion lässt sich folgendermaßen beschreiben:

$$\mathbf{q} = \left(\hat{v}_x \cdot \sin \frac{\phi}{2}, \hat{v}_y \cdot \sin \frac{\phi}{2}, \hat{v}_z \cdot \sin \frac{\phi}{2}, \cos \frac{\phi}{2} \right) \quad (2.15)$$

Eingesetzt ergibt sich:

$$\mathbf{q} = \left(\frac{r_x}{\phi} \sin \frac{\phi}{2}, \frac{r_y}{\phi} \sin \frac{\phi}{2}, \frac{r_z}{\phi} \sin \frac{\phi}{2}, \cos \frac{\phi}{2} \right) \quad (2.16)$$

Mit Hilfe der Gleichungen 2.13 und 2.16 lässt sich die Umrechnung von Rodrigues-Parametern in ein Quaternion durchführen.

2.3 Transformation von Koordinatensystemen

Um die genaue Position des UAVs bestimmen zu können, sind mehrere Koordinatentransformationen notwendig. Transformationen sind sowohl bei der Lokalisierung im Raum als auch bei der extrinsischen Kalibrierung der Kameras erforderlich. Es ist möglich, einen Punkt ${}^a\mathbf{r}$, ausgedrückt relativ zum Koordinatensystem a , relativ zum Koordinatensystem b auszudrücken, sofern Position und Orientierung des Systems a relativ zum System b bekannt sind. Ziel ist es, aus dieser Beziehung eine Transformationsmatrix abzuleiten, die zugleich die Rotation und die Translation zwischen den beiden Systemen beschreibt. [9, S. 16–17]. Die Orientierung des Systems a relativ zum System b wird durch eine Rotationsmatrix ${}^b\mathbf{R}_a$ beschrieben. Die Lage des Ursprungs von a relativ zu b wird durch den Translationsvektor ${}^b\mathbf{t}_a \in \mathbb{R}^3$ angegeben. Ein im System a ausgedrückter Punkt ${}^a\mathbf{r}$ lässt sich dann mittels

$${}^b\mathbf{r} = {}^b\mathbf{R}_a {}^a\mathbf{r} + {}^b\mathbf{t}_a \quad (2.17)$$

in das System b überführen. Diese Vorschrift kombiniert eine Drehung und eine anschließende Verschiebung in einem einzigen Schritt. Durch Einführung homogener Koordinaten ${}^a\tilde{\mathbf{r}} = \begin{bmatrix} {}^a\mathbf{r}^T & 1 \end{bmatrix}^T$ lässt sich die Drehung und Verschiebung in eine einzige Matrix überführen. Die zugehörige *homoge-*

ne Transformationsmatrix ${}^b\mathbf{T}_a \in \text{SE}(3)$ ist definiert als

$${}^b\mathbf{T}_a = \begin{bmatrix} {}^b\mathbf{R}_a & {}^b\mathbf{t}_a \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}. \quad (2.18)$$

Damit lässt sich Gleichung (2.17) kompakt schreiben als

$${}^b\tilde{\mathbf{r}} = {}^b\mathbf{T}_a {}^a\tilde{\mathbf{r}}. \quad (2.19)$$

Die Rotationsmatrix ${}^b\mathbf{R}_a$ kann dabei aus einem Einheitsquaternion $\mathbf{q} = (w, x, y, z)$ berechnet werden:

$$R(\mathbf{q}) = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2zw & 2xz + 2yw \\ 2xy + 2zw & 1 - 2x^2 - 2z^2 & 2yz - 2xw \\ 2xz - 2yw & 2yz + 2xw & 1 - 2x^2 - 2y^2 \end{bmatrix}. \quad (2.20)$$

Eine wesentliche Eigenschaft homogener Transformationsmatrizen ist ihre Verkettbarkeit. Sind die Transformationen ${}^c\mathbf{T}_b$ (von b nach c) und ${}^b\mathbf{T}_a$ (von a nach b) bekannt, so ergibt sich die direkte Transformation von a nach c durch

$${}^c\mathbf{T}_a = {}^c\mathbf{T}_b {}^b\mathbf{T}_a. \quad (2.21)$$

2.4 Sensorsysteme

Zum besseren Verständnis der Arbeit werden in diesem Unterkapitel die relevanten Sensortypen vorgestellt.

Die zentrale Sensoreinheit für ein UAV ist die Inertial Measurement Unit (IMU), welche Beschleunigungen und Winkelgeschwindigkeiten misst. Die IMU besteht typischerweise aus einem 3-Achsen-Beschleunigungssensor und einem 3-Achsen-Gyroskop. In einigen Fällen kann auch ein 3-Achsen-Magnetometer integriert sein, um die Orientierung relativ zum Erdmagnetfeld zu bestimmen. Ein weiterer Sensortyp ist das Barometer zur Bestimmung der Höhe, indem es den aktuellen Luftdruck misst. Weiterhin kann auch die Motordrehzahl oder Motorlast als Sensorinformation genutzt werden, um Rückschlüsse auf den aktuellen Bewegungszustand zu ziehen [8]. All diese Sensoren liefern nicht direkt die Position oder Orientierung des Roboters, reagieren jedoch schnell auf Bewegungsänderungen. Jeder dieser Sensoren ist mit einer gewissen Messunsicherheit behaftet, die durch Rauschen, Kalibrierungsfehler oder Umwelteinflüsse unterschiedlich stark ausgeprägt sein kann. Diese Messunsicherheiten lassen sich grundsätzlich in deterministische und stochastische Fehleranteile unterteilen [13]. Ein wesentlicher deterministischer Fehler ist der Bias, ein systematischer Messfehler in Form eines konstanten Versatzes, der die Messung auch dann verfälscht, wenn keine physikalische Größe anliegt. So gibt beispielsweise ein ruhendes Gyroskop trotz fehlender Drehung einen von null verschiedenen Wert aus. Dem gegenüber steht das Rauschen als stochastischer, mittelwertfreier Fehleranteil. Die Sensorfusion, also die Kombination der Daten mehrerer Sensoren, ermöglicht es, die Genauigkeit der Messungen zu verbessern [14].

Die Integration dieser Sensordaten über die Zeit ermöglicht es, die Bewegung des Roboters zu verfolgen und eine Schätzung seines Zustands zu erhalten, was als Odometrie bezeichnet wird. Da die

Sensoren, wie zuvor beschrieben, trotz Fusion fehlerbehaftet sind, führt die Integration über die Zeit zu einer zunehmenden Unsicherheit. Die Odometrie eines Flugroboters ist besonders herausfordernd, da sich das UAV in drei Dimensionen frei bewegt. Der zunehmende Fehler der Odometrie kann durch die Verwendung von externen Referenzen, wie Global Positioning System (GPS), korrigiert werden.

2.5 Extended Kallman Filter

In dieser Arbeit wird nicht näher auf die Sensorfusion eingegangen, da der Extended Kalman Filter (EKF) bereits vollständig im Flugcontroller implementiert ist. Auch die speziell im Pixhawk (PX4) zugrundeliegenden mathematischen Modelle werden nicht weiter erläutert. Zum Verständnis der Positionsbestimmung sowie der Parametrierung des EKF ist es jedoch notwendig, die Grundlagen des Kalman-Filters zu kennen.

Ein Kalman-Filter ist ein Algorithmus zur Schätzung des Zustands eines dynamischen Systems durch die Kombination von Vorhersagen aus einem mathematischen Modell und Messungen von Sensoren [15]. Der EKF ist ein rekursiver Filter, wodurch er kontinuierlich neue Messwerte verarbeiten kann. Im Folgenden wird der grundlegende Aufbau des Filters geschildert. Der Filter besteht im Wesentlichen aus drei Phasen. In der Literatur wird zwischen zwei Arten von Zustandsschätzungen unterschieden: Der *Prior* $(\cdot)^-$ bezeichnet die durch das Systemmodell vorhergesagte Schätzung vor der Einbeziehung einer neuen Messung. Der *Posterior* $(\cdot)^+$ bezeichnet die korrigierte Schätzung nach der Einbeziehung der Messung [16]. Der Operator $(\hat{\cdot})$ kennzeichnet dabei stets eine Schätzgröße – $\hat{x}$ ist demnach eine Schätzung des wahren Zustands x [16].

Zu Beginn muss eine Anfangsbedingung festgelegt werden. Hierfür benötigt das System einen initialen Zustandsschätzwert $\hat{x}_0^+$ sowie die zugehörige Unsicherheit in Form der Zustandskovarianzmatrix P_0^+ [16]. In der Prädiktionsphase wird der Zustand des Systems zum nächsten Zeitpunkt auf Basis eines mathematischen Modells berechnet. Der Zustand $\hat{x}_k^-$ wird dabei aus dem vorherigen Zustand $\hat{x}_{k-1}^+$ und der Steuereingabe u_{k-1} über die nichtlineare Modellfunktion f prädiziert:

$$\hat{x}_k^- = f(\hat{x}_{k-1}^+, u_{k-1}) \quad (2.22)$$

Gleichzeitig wird die Zustandskovarianzmatrix P_k^- fortgeschrieben, die die Unsicherheit der Schätzung beschreibt. Sie wächst in dieser Phase durch das Prozessrauschen Q_{k-1} an:

$$P_k^- = F_{k-1} P_{k-1}^+ F_{k-1}^T + Q_{k-1} \quad (2.23)$$

Dabei ist F_{k-1} die Jacobi-Matrix der Modellfunktion f , die durch Linearisierung um den aktuellen Schätzwert gebildet wird. Das Prozessrauschen Q beschreibt, wie stark das Systemmodell vom realen Verhalten abweicht: Ein hoher Wert bedeutet ein geringeres Vertrauen in das Modell.

In der Korrekturphase wird der prädizierte Zustand auf Basis eines Messwerts z_k korrigiert. Zunächst wird der Kalman-Gain K_k berechnet, der bestimmt, wie stark die Messung in die Schätzung einfließt:

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \quad (2.24)$$

Hierbei ist $\mathbf{H}_k$ die Jacobi-Matrix der Messfunktion h und $\mathbf{R}_k$ die Messrauschkovarianzmatrix. $\mathbf{R}$ beschreibt die Unsicherheit der Sensormessungen: Ein hoher Wert führt zu einem geringeren Vertrauen in die Messung und damit zu einer schwächeren Korrektur. Der Kalman-Gain gewichtet folglich Modellvorhersage und Messung anhand ihrer jeweiligen Unsicherheiten.

Anschließend wird der korrigierte Zustand $\hat{\mathbf{x}}_k^+$ aus dem prädizierten Zustand und der gewichteten Differenz zwischen tatsächlicher Messung $\mathbf{z}_k$ und erwarteter Messung $h(\hat{\mathbf{x}}_k^-)$ – der sogenannten Innovation – berechnet:

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \left(\mathbf{z}_k - h(\hat{\mathbf{x}}_k^-) \right) \quad (2.25)$$

Abschließend wird die Zustandskovarianzmatrix aktualisiert und durch die Einbeziehung der Messung reduziert:

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- \quad (2.26)$$

Die aktualisierten Größen $\hat{\mathbf{x}}_k^+$ und $\mathbf{P}_k^+$ dienen im nächsten Zeitschritt wiederum als Ausgangswerte der Prädiktionsphase, wodurch sich der rekursive Charakter des Filters ergibt [16].

2.6 Flugcontroller Pixhawk 6X

PX4 ist ein Open-Source-Autopilot-Flugstack, der die Flugsteuerung, Zustandsschätzung und Kommunikation eines UAV übernimmt. Das Pixhawk-Projekt wurde ursprünglich als studentisches Projekt an der ETH Zürich gestartet [17] und hat sich seitdem zu einem weit verbreiteten Open-Source-Standard in der UAV-Forschung und -Industrie entwickelt. Der in dieser Arbeit verwendete Pixhawk 6X basiert auf dem Pixhawk FMUv6X-Standard und wurde von Holybro in Zusammenarbeit mit dem PX4-Entwicklerteam entwickelt.[18] Er bildet das zentrale Steuerungselement des verwendeten UAV.

Der Flugcontroller besteht aus zwei Teilen: dem Baseboard V2A und dem FMU-6X-Modul. Das Flight Management Unit (FMU)-Modul wird auf das Baseboard aufgesteckt und verschraubt. Die Komponenten sind in Abbildung 2.3 dargestellt. Im weiteren Verlauf der Arbeit wird dieser Zusammenschluss als Flugcontroller bezeichnet. Das FMU stellt die zentrale Recheneinheit des Flugcontrollers dar, auf der der Hauptprozessor (STM32H753) sowie die PX4-Firmware betrieben werden. Das Baseboard stellt vor allem die Schnittstellen zu Sensoren, anderen Komponenten und der Stromversorgung bereit.

FMUv6X-Modul

Baseboard V2A



Abbildung 2.3: Komponenten des Pixhawk 6X: FMU-Modul und Baseboard V2A [3]

2.6.1 Sensorik

Der Pixhawk 6X verfügt über eine dreifach redundante IMU sowie zwei redundante Barometer, die auf getrennten Bussen mit unabhängiger Stromversorgung betrieben werden. Diese Redundanz erhöht die Zuverlässigkeit: Erkennt der PX4 den Ausfall eines Sensors, schaltet das System auf einen funktionierenden Sensor um, um die Flugsteuerung aufrechtzuerhalten.[18] Die Sensoren sind, wie in Abschnitt ?? beschrieben, mit einer Messunsicherheit behaftet, weshalb die redundante Auslegung und die anschließende Sensorfusion im EKF erfolgen.

Die in der eingesetzten Revision verbauten Sensoren sind in Tabelle ?? aufgeführt.

Sensortyp	Bauteil
Beschleunigung/Gyroskop	ICM-20649 / BMI088
Beschleunigung/Gyroskop	ICM-42688-P
Beschleunigung/Gyroskop	ICM-42670-P
Barometer	2× BMP388
Magnetometer	BMM150

Tabelle 2.1: Sensorbestückung des Pixhawk 6X [18]

Zusätzlich sind die IMUs vibrationsisoliert gelagert und werden über Heizwiderstände auf einer konstanten Betriebstemperatur gehalten, um temperaturbedingte Messabweichungen zu reduzieren.[18] Ein Magnetometer ist zwar verbaut, wird in dieser Arbeit jedoch nicht zur Lagebestimmung verwendet (siehe Abschnitt 2.6.3).

2.6.2 Schnittstellen

Der Flugcontroller stellt über das Baseboard eine Vielzahl von Schnittstellen zur Anbindung externer Komponenten bereit. Für diese Arbeit sind insbesondere die folgenden Schnittstellen relevant:

Die serielle Schnittstelle (UART) der Telemetrie-Ports dient der Kommunikation zwischen dem Flugcontroller und dem Jetson Orin Nano, über welche die extern berechnete Positionsschätzung

an den EKF übermittelt wird. Die genaue Anbindung wird in Kapitel ?? beschrieben. Über den USB-Anschluss erfolgt die Parametrierung und Kalibrierung des Flugcontrollers mittels der Bodenstationssoftware QGroundControl. Die PWM-Ausgänge des Baseboards steuern die Elektromotoren des UAV an. Darüber hinaus stehen weitere Schnittstellen wie CAN, I²C und SPI zur Anbindung zusätzlicher Sensoren und Peripherie zur Verfügung, die in dieser Arbeit jedoch nicht verwendet werden.[18]

2.6.3 EKF2 – Zustandsschätzung und Sensorfusion

Die Zustandsschätzung des PX4 erfolgt über den Extended Kalman Filter 2 (EKF2), dieser ist in 2.5 beschriebenen EKF. Der EKF2 fusioniert die Messungen der bordeigenen Sensoren mit optionalen externen Referenzen, um Lage, Geschwindigkeit und Position des UAV zu schätzen. Der Zustandsvektor des EKF2 umfasst 24 Zustände, darunter die Lage als Quaternion, Geschwindigkeit, Position, die Bias-Werte von Gyroskop und Beschleunigungssensor, das Erdmagnetfeld, den Magnetometer-Bias sowie die horizontale Windgeschwindigkeit.[19] Da in dieser Arbeit kein GPS zur Verfügung steht, wird die Position des UAV über eine externe visuelle Positionsschätzung bereitgestellt. Diese wird als externe Referenz in den EKF2 eingespeist und ersetzt dort die Funktion, die andernfalls vom GPS übernommen würde. Das Verhalten des EKF2 lässt sich über eine Reihe von Parametern konfigurieren, welche unter anderem festlegen, welche Sensorquellen zur Positions- und Höhenbestimmung herangezogen werden. Die konkrete Parametrierung für den GPS-freien Betrieb mit externer Positionsreferenz wird in Kapitel ?? erläutert.

2.6.4 Betriebssystem

Der Pixhawk 6X betreibt auf dem Hauptprozessor STM32H753 das Echtzeitbetriebssystem NuttX, auf dem die PX4-Firmware ausgeführt wird. NuttX ist ein Echtzeitbetriebssystem, das sicherstellt, dass zeitkritische Steuerungsaufgaben innerhalb garantierter Zeitschranken ausgeführt werden. [20]

Die Kommunikation mit externen Systemen in dieser Arbeit erfolgt über das MAVLink-Protokoll. MAVLink ist ein leichtgewichtiges Nachrichtenprotokoll, das speziell für die Kommunikation zwischen Flugcontrollern und Bodenstationen bzw. Companion-Computern entwickelt wurde. Es serialisiert Telemetriedaten, Steuerbefehle und Sensorwerte in kompakte Binärpakete, die über serielle Schnittstellen oder User Datagram Protocol (UDP) übertragen werden. [21], [22]

Zur Analyse des Flugverhaltens erzeugt der PX4 während des Betriebs ULog-Dateien, die auf der internen SD-Karte gespeichert werden. Diese Log-Dateien enthalten zeitgestempelte Aufzeichnungen aller relevanten Zustandsgrößen, Sensorwerte und EKF2-internen Größen und ermöglichen eine detaillierte Nachanalyse des Systemverhaltens nach dem Flug. [23], [24]

2.6.5 Random Sample Consensus (RANSAC)

Der RANSAC ist ein iteratives Verfahren zur robusten Schätzung von Modellparametern aus einem Datensatz, der einen erheblichen Anteil an Ausreißern enthält. Der RANSAC unterscheidet sich von klassischen Ausgleichsverfahren dadurch, dass es nicht versucht, alle Datenpunkte gleichzeitig zu berücksichtigen [25]. Stattdessen wird das Modell aus möglichst kleinen, zufällig gewählten Teilmen-

gen bestimmt und anhand der übrigen Daten bewertet. Dadurch bleibt die Schätzung auch dann zuverlässig, wenn ein großer Teil der Daten fehlerhaft ist [25].

Die Grundidee besteht darin, wiederholt eine minimale Teilmenge von Datenpunkten auszuwählen und daraus ein Modell zu berechnen. Die Größe dieser Teilmenge entspricht der minimalen Anzahl an Punkten, die zur eindeutigen Bestimmung der Modellparameter erforderlich ist. Für jedes so berechnete Modell wird anschließend geprüft, wie viele der übrigen Datenpunkte mit ihm übereinstimmen. Ein Datenpunkt gilt als konsistent, wenn sein Abstand zum Modell einen festgelegten Schwellenwert t unterschreitet:

$$d(\mathbf{x}_i, \mathcal{M}) < t, \quad (2.27)$$

wobei $d(\mathbf{x}_i, \mathcal{M})$ den Abstand des Punktes $\mathbf{x}_i$ zum Modell $\mathcal{M}$ bezeichnet. Die Menge aller konsistenten Punkte wird als Konsensmenge bezeichnet, ihre Elemente als Inlier; die übrigen Punkte gelten als Ausreißer [25].

2.6.6 Robot Operating System 2 (ROS2)

ROS2 ist ein Open-Source-Framework für die Entwicklung von Robotersoftware. Es stellt eine einheitliche Middleware bereit, über die verteilte Software-Komponenten – sogenannte *Nodes* – miteinander kommunizieren können. wodurch kein zentraler Master-Prozess mehr benötigt wird und eine höhere Robustheit sowie Echtzeitfähigkeit erreicht werden. In dieser Arbeit wird ROS2 Humble sowie ROS2 Jazzy eingesetzt.[26]

2.6.7 Nodes und Topics

Die grundlegende Kommunikationseinheit in ROS2 ist der *Node*: ein eigenständiger Prozess, der eine spezifische Aufgabe übernimmt, beispielsweise die Kamerabildverarbeitung. Nodes tauschen Daten über *Topics* nach dem Publish-Subscribe-Prinzip aus. Ein Node, der Daten erzeugt, veröffentlicht diese als *Publisher* auf einem Topic; ein oder mehrere *Subscriber*-Nodes empfangen die Nachrichten asynchron. Topics sind typisiert, das heißt jedes Topic überträgt Nachrichten eines fest definierten Nachrichtentyps, beispielsweise `geometry_msgs/PoseStamped` für Positionsdaten oder `sensor_msgs/Image` für Kamerabilder. [27], [28]

2.6.8 Bag-Dateien

ROS2 ermöglicht die Aufzeichnung von Topic-Daten in sogenannten *Bag-Dateien* mittels des Werkzeugs `rosbag2`. Aufgezeichnete Bags können anschließend offline wiedergegeben werden, sodass das Systemverhalten ohne erneuten Flug reproduzierbar analysiert werden kann. [29]

2.6.9 TF2 – Koordinatentransformationen

Das `tf2`-Paket verwaltet in ROS2 einen baumförmigen Graphen von Koordinatentransformationen zwischen den im System definierten Referenzrahmen.**ROS2Humble** Jede Transformation beschreibt die räumliche Beziehung zwischen zwei Frames als homogene Transformation bestehend aus Trans-

lation und Rotation. Der TF-Baum wird kontinuierlich über die Topics `/tf` und `/tf_static` aktualisiert, wobei statische Transformationen – beispielsweise die feste Lage einer Kamera relativ zum UAV-Körper – über `/tf_static` einmalig veröffentlicht werden. Jeder Node kann zu jedem Zeitpunkt die Transformation zwischen zwei beliebigen Frames abfragen, sofern ein zusammenhängender Pfad im TF-Baum existiert.

3 Stand der Technik

Dieses Kapitel gibt einen Überblick über den aktuellen Stand der Technik im Bereich der Global Navigation Satellite System (GNSS)-freien Lokalisierung von UAV und ordnet die in dieser Arbeit gewählten Methoden in den wissenschaftlichen Kontext ein. Zunächst werden bestehende Lokalisierungsansätze ohne GNSS vorgestellt und hinsichtlich ihrer Eignung für Innenraumanwendungen bewertet. Anschließend werden verschiedene Fiducial-Marker-Systeme verglichen und deren Eigenschaften im Hinblick auf die Anforderungen dieser Arbeit diskutiert. Abschließend wird auf die simultane Lokalisierung und Kartierung eingegangen, die die Grundlage für die in dieser Arbeit verwendete Kartierungsmethodik bildet.

3.1 GNSS-freie Lokalisierung von UAVs

Die Navigation ohne GNSS stellt in der Luftfahrt eine erhebliche Herausforderung dar, da GNSS als primäre Quelle absoluter Positionsinformationen dient. In dessen Abwesenheit muss die Position aus der Integration fehlerbehafteter Inertialmessungen geschätzt werden, was, wie in Abschnitt ?? beschrieben, zu einem zeitlich anwachsenden Positionsfehler führt. Daher ist es notwendig, diesen Fehler durch absolute oder relative Lokalisierungsmethoden auszugleichen.[30]

3.1.1 Absolute Lokalisierung

Absolute Lokalisierungsverfahren spielen eine wichtige Rolle, wenn die genaue Position eines UAV bezogen auf einen globalen Referenzrahmen bekannt sein muss.[30] Die gängigste Methode zur Umsetzung einer absoluten Lokalisierung ist ein satellitengestütztes System wie GPS oder Galileo. Da in der für diese Arbeit relevanten Flugumgebung kein ausreichender Satellitenempfang herrscht, finden diese Systeme keine Anwendung. Eine weitere Möglichkeit der absoluten Lokalisierung bieten Template- und Feature-basierte Verfahren sowie Semantic Mapping.

Bei Template- beziehungsweise Feature-basierten Verfahren werden Aufnahmen des UAV mit bekannten Referenzbildern oder gespeicherten Merkmalen verglichen. Diese Systeme berücksichtigen ausschließlich visuelle Merkmale, ohne semantische Beziehungen einzubeziehen.[30] Ein verwandtes Konzept stellen Fiducial-Marker dar, bei denen anstelle natürlicher Bildmerkmale künstlich erzeugte, eindeutig kodierte Muster als Referenz dienen. Durch ihre definierte Geometrie und Kodierung ermöglichen sie eine präzisere und robustere Posenschätzung als natürliche Bildmerkmale, weshalb sie in dieser Arbeit als Grundlage der Lokalisierung eingesetzt werden. Im Sinne einer globalen absoluten Lokalisierung im Weltkoordinatensystem sind Template- und Feature-basierte Verfahren für Innenraumanwendungen jedoch wenig geeignet, da geeignete Referenzdaten schwer zu erfassen sind.[30]

Semantic Mapping verfolgt einen ähnlichen Ansatz: Auch hier werden zunächst Referenzdaten erstellt, die anschließend mit aktuell erfassten Daten abgeglichen werden. Der Unterschied liegt darin, dass nicht nur visuelle Merkmale, sondern auch semantische Beziehungen berücksichtigt werden, so wird beispielsweise ein Tisch als Tisch und eine Treppe als Treppe interpretiert.[30] Dieses Verfahren findet in der vorliegenden Arbeit keine Anwendung, da die Erkennungsleistung stark von der Stabilität der jeweiligen Umgebung abhängig ist.

Da die einzige praktikabel umsetzbare Methode der absoluten Lokalisierung ein satellitengestütztes System darstellt, welches wie bereits erwähnt in der vorliegenden Umgebung nicht verfügbar ist, wird in dieser Arbeit auf absolute Lokalisierungsverfahren verzichtet und ausschließlich relative Lokalisierungsmethoden betrachtet. Die grundlegende Methodik der zuvor vorgestellten Verfahren findet dabei ebenfalls Anwendung, jedoch ohne Bezug zu einem globalen Referenzrahmen.

3.1.2 Relative Lokalisierung

Die relative Lokalisierung zielt darauf ab, die Position und Orientierung eines UAV innerhalb eines lokalen Koordinatensystems zu bestimmen. Die Position wird dabei bezogen auf einen definierten Referenzpunkt ermittelt. Im Folgenden werden die gängigsten Methoden vorgestellt und hinsichtlich ihrer Eignung für die vorliegende Innenraumanwendung bewertet.

3.1.2.1 Dead Reckoning Localization

Bei der Dead Reckoning Localization (DRL) handelt es sich um ein Verfahren, bei dem die aktuelle Position eines Objekts ausgehend von einem letzten bekannten Startpunkt kontinuierlich berechnet wird.[31] Die Berechnung erfolgt auf Basis der bereitgestellten IMU-Daten durch Integration dieser. Wie in Abschnitt 2.4 bereits beschrieben, wirken auf die Sensoren erhebliche Fehler, sodass die direkte Integration der rohen IMU-Messdaten zu großen Translationsfehlern führt. Aus diesem Grund ist eine Filterung der Eingangsdaten notwendig. Am weitesten verbreitet sind hierbei verschiedene Abwandlungen des Kalman-Filters.

Experimentelle Untersuchungen mit dem PX4-Autopiloten und einem handelsüblichen Pixhawk-Flugregler zeigen, dass der EKF2 bereits nach 7 Sekunden eine Geschwindigkeitsabweichung von $\sigma_{vel} \approx 0,024$ m/s aufweist, was zu einem zunehmenden Positionsfehler im Dezimeterbereich führt.[32] Verschiedene Arbeiten zeigen, dass sich der Drift vor allem durch Nutzung verbesserter Algorithmen wie dem Invariant Extended Kalman Filter (IEKF) minimieren, jedoch nicht vollständig vermeiden lässt.[31][32][19] Da im verwendeten Flugcontroller, wie in Kapitel 2.6 beschrieben, der EKF implementiert ist, wird nicht weiter auf andere Abwandlungen des Algorithmus eingegangen.

Grundsätzlich ist DRL für den Innenraum nur zeitlich begrenzt geeignet und als primäre Positionsquelle nicht ausreichend. Im Normalfall wird DRL daher als Fallback-Strategie genutzt, wenn die primäre Positionsquelle nicht verfügbar ist.[32] In dieser Arbeit wird der Flugcontroller entsprechend konfiguriert.

3.1.2.2 Visuelle Odometrie und optischer Fluss

Visuelle Odometrie sowie optischer Fluss sind grundlegende Verfahren zur Lokalisierung eines UAV. Die Visuelle Odometrie (VO) schätzt die Bewegung einer Kamera anhand sequenzieller Bilddaten, ohne eine Karte der Umgebung zu erstellen. Dabei werden markante Bildpunkte, auch Landmarken genannt, zwischen aufeinanderfolgenden Frames verfolgt und aus deren Verschiebung auf die Kamerabewegung geschlossen.[30]

Darüber hinaus existieren Visuelle Inertial Odometrie (VIO)-Ansätze, die die Robustheit der VO durch Einbeziehen von IMU-Informationen verbessern.[33] Die visuellen Daten können dabei den Drift der IMU reduzieren, während die IMU-Daten den metrischen Maßstab sowie Roll- und Nickwinkel

aus den visuellen Daten wiederherstellen können.[33] Es gibt sowohl mono- als auch stereokamerabasierte Ansätze, wobei Stereokameras aufgrund der direkten Tiefenschätzung eine höhere Genauigkeit erzielen.[33] Die Fusion von IMU und VO zu VIO erfolgt typischerweise über einen EKF.[33]

Da VO und VIO keinen absoluten Positionsbezug besitzen, akkumulieren diese Verfahren über die Zeit einen Drift und sind damit nur bedingt für eine langfristig stabile Innenraumlokalisierung geeignet.[30] Zudem ist die Leistung kamerabasierter Verfahren stark von den Lichtverhältnissen der Umgebung abhängig.[33] Weiterhin visuelle Odometrie setzt strukturierte Umgebungen voraus, welche in der Versuchsumgebung nicht ohne weiteres gegeben sind.

Der optische Fluss stellt ein verwandtes Verfahren dar, das im Gegensatz zur VO lediglich zweidimensionale Geschwindigkeitsinformationen liefert.[30] Aus diesem Grund ist der optische Flow nicht zur Positionserkennung einer UAV geeignet.

3.1.3 Simultane Lokalisierung und Kartierung

Simultaneous Localization and Mapping (SLAM) beschreibt eine Problemstellung an der Schnittstelle von Computer Vision und Robotik. Dabei geht es um die selbstständige Erstellung einer Karte einer unbekannten Umgebung bei gleichzeitiger Bestimmung der eigenen Position innerhalb dieser Umgebung.[34] Der Begriff simultan beschreibt genau diesen Zusammenhang: Kartierung und Positionsbestimmung müssen parallel erfolgen, da jede Aufgabe die andere voraussetzt.[34] Die Lösung dieses Problems erfolgt üblicherweise durch filterbasierte Methoden, wie den EKF, oder durch optimierungsbasierte Methoden, die im Folgenden näher beschrieben werden.[34]

Filterbasierte SLAM Algorithmen verarbeiten Messungen sequenziell und lassen keine Korrektur von vergangenen Messungen zu. Insbesondere fehlt filterbasierten Methoden die Fähigkeit zur globalen Kartenkorrektur durch Loop Closure, was zu einer geringeren Kartierungsgenauigkeit im Vergleich zu optimierungsbasierten Methoden führt. In der Lokalisierung sind filterbasierte Methoden aufgrund ihrer geringeren Rechenkomplexität jedoch schneller.[34] In der aktuellen Literatur dominieren optimierungsbasierte Methoden, und neuere Arbeiten orientieren sich zunehmend an diesem Ansatz.

3.1.3.1 Phasen optimierungsbasierter SLAM Methoden

Der allgemeine optimierungsbasierte SLAM-Prozess lässt sich in folgende Phasen unterteilen:

Feature Extraction bezeichnet die Extraktion markanter Merkmale aus den Sensordaten, die als Grundlage für die spätere Zuordnung dienen.[34]

Feature Matching beschreibt den Abgleich extrahierter Merkmale zwischen aufeinanderfolgenden Frames, um die Bewegung des Systems zu schätzen.[34] Beispiele dieser Methoden werden in Abschnitt 3.1.2 beschrieben.

Map Initialization bezeichnet die Initialisierung der Karte zu Beginn des SLAM-Prozesses. Dabei werden zunächst markante Bildmerkmale aus den ersten Frames extrahiert und einander zugeordnet, um die initiale Struktur der Umgebung zu schätzen. Aus diesen Zuordnungen werden erste Kartenpunkte sowie deren räumliche Positionen im dreidimensionalen Raum bestimmt. Die Map Initialization bildet die Grundlage, auf der alle weiteren Kartenpunkte und Posenschätzungen aufgebaut werden. [35]

Loop Closure bezeichnet die Erkennung bereits besuchter Orte. Wird ein solcher Ort erkannt,

wird die akkumulierte Abweichung anhand gespeicherter Merkmale berechnet und als zusätzlicher Constraint den Faktorgraphen eingetragen. Die anschließende globale Optimierung verbessert die Konsistenz der gesamten Karte erheblich. [34] Der schematische Ablauf von Loop Closure ist in Abbildung 3.1 dargestellt. Die Funktion des Faktorgraphen wird in Abschnitt 3.1.3.2 näher beschrieben.

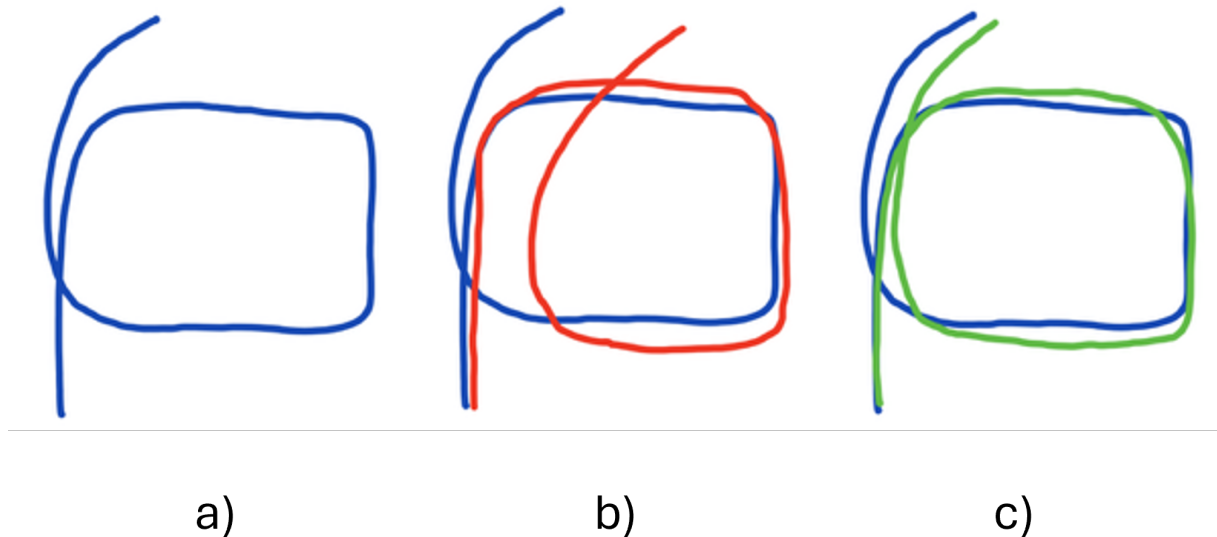


Abbildung 3.1: Veranschaulichung des Loop-Closure-Effekts: (a) Reale Trajektorie ohne akkumulierten Fehler. (b) Geschätzte Trajektorie mit akkumuliertem Drift ohne Loop Closure. (c) Korrigierte Trajektorie nach Anwendung von Loop Closure mit reduziertem akkumuliertem Drift. vgl. [36]

3.1.3.2 Faktorgraph Optimierung

Im folgenden Abschnitt wird der Faktorgraph nur schematisch erläutert, um ein grundsätzliches Verständnis dieser Methodik zu erlangen. Genaue Herleitungen sowie die Mathematik hinter dem Prinzip können den Quellen [37], [38] und [39] entnommen werden.

Ein Faktorgraph ist eine grafische Darstellung eines Schätzproblems und besteht aus zwei Arten von Knoten: Variablenknoten und Faktorknoten.[37] Die Variablenknoten sind die unbekannten Größen, die geschätzt werden sollen – bei SLAM die Pose des UAV und die Position der Landmarken.[37] Die Faktorknoten verbinden diese Variablen und repräsentieren die Messungen oder das Vorwissen.[37]

Der Faktorgraph in Abbildung 3.2 zeigt die grundlegende Struktur eines SLAM-Problems. Hierbei ist der Faktorgraph stark vereinfacht; auf mathematische Beziehungen wird verzichtet. Die blauen Kreise repräsentieren die Variablenknoten der UAV-Posen x_1 bis x_4 , also die gesuchten Positionen und Orientierungen des UAV zu aufeinanderfolgenden Zeitpunkten. Die grünen Kreise t_1 bis t_3 stellen die Landmarken dar, deren Positionen im Weltkoordinatensystem ebenfalls als Variablenknoten geschätzt werden.

Es werden zwei Arten von Faktorknoten (Constraints) unterschieden: Die violetten Quadrate repräsentieren Odometrie-Constraints, die jeweils zwei aufeinanderfolgende Posen verbinden und die relative Bewegung zwischen diesen beschreiben. Die orangen Quadrate sind Messungs-Constraints

und verbinden eine UAV-Pose mit einer Landmarke – sie repräsentieren die Beobachtung, dass Tag t_i von Pose x_j aus detektiert wurde.

Wird die Pose x_4 als bereits bekannter Ort – identisch mit x_1 – wiedererkannt, entsteht ein Loop-Closure-Constraint, der x_1 und x_4 im Faktographen verbindet, wie durch den roten Pfeil in Abbildung 3.2 dargestellt. Dieser zusätzliche Constraint schließt die Schleife und ermöglicht dem Optimierer, den über die Trajektorie akkumulierten Drift rückwirkend über alle Posen zu korrigieren.[40]

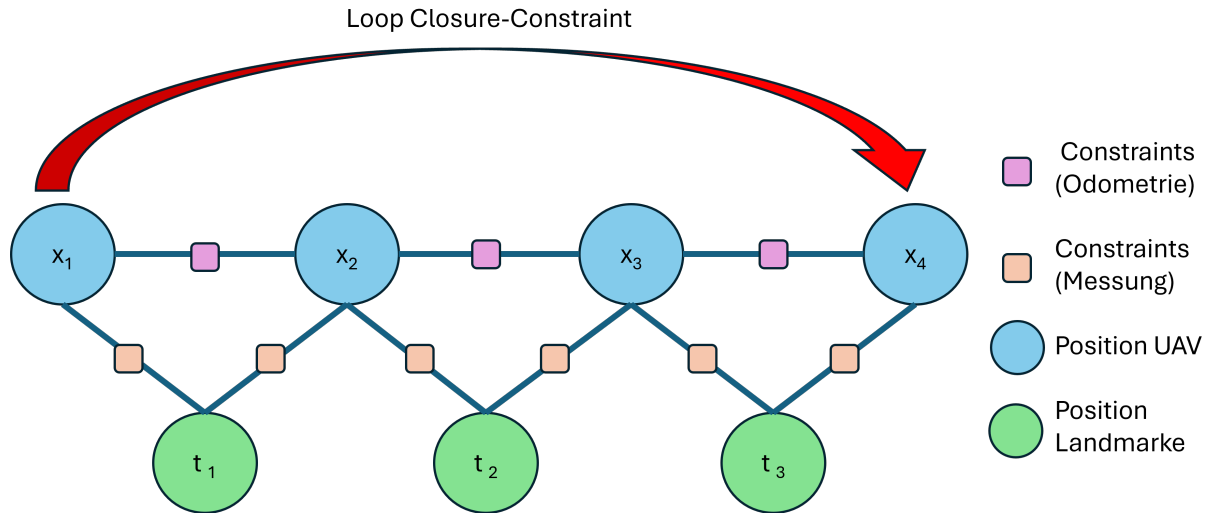


Abbildung 3.2: Vereinfachter Faktograph eines SLAM-Problems, vgl. [37], [38], [39]

Das Ziel der Optimierung ist die Maximum-a-posteriori-Schätzung – also die Konfiguration aller Variablen (Posen und Landmarken) zu finden, die die Gesamtwahrscheinlichkeit aller Beobachtungen maximiert. Äquivalent dazu wird der Gesamtfehler über alle Constraints minimiert.[41] Bildlich bedeutet das, dass der Optimierer die Variablen solange “verschiebt”, bis der Gesamtfehler über allen Messungen minimal ist. Bekannte Faktograph Bibliotheken sind Georgia Tech Smoothing and Mapping (GTSAM) und General Graph Optimization (g2o).

3.1.3.3 SLAM-Varianten

Kamerabasiertes Visual SLAM (vSLAM) nutzt visuelle Sensordaten, um gleichzeitig die Pose eines Roboters zu schätzen und eine Karte der Umgebung zu erstellen.[34] Es baut auf den Konzepten der VO und VIO auf und erweitert diese um die Fähigkeit zur globalen Kartenkonsistenz durch Loop Closure.[39] Durch die Erkennung bereits besuchter Orte kann das System akkumulierte Posenfehler korrigieren und eine global konsistente Karte aufbauen. Ein weit verbreitetes Beispiel ist orb-slam3, ein Open-Source-System, das ORB-Merkmale zur Merkmalerkennung verwendet.[34] Je nach Konfiguration unterstützt es Mono-, Stereo- und RGB-D-Kameras als reines vSLAM sowie die Kombination mit einer IMU als visuell-inertialen SLAM.

TagSLAM ist eine Spezialisierung von vSLAM für Fiducial-Marker. Anstelle natürlicher Bildmerkmale werden AprilTag-Marker als definierte Referenzpunkte verwendet. Da die Geometrie der Marker bekannt ist, entfällt die aufwendige Merkmalsbeschreibung und das Matching; stattdessen liefert jede Markerdetektion eine direkte Posenmessung relativ zur Kamera. TagSLAM nutzt den GTSAM zur Faktograph Optimierung als Backend und ermöglicht neben vollständigem SLAM Posenschätzung.[42]

Light Detection and Ranging (LiDAR)-SLAM verwendet einen Laserscanner anstelle einer Kamera als primären Sensor [43]. Ein LiDAR misst Entfernungen durch Laufzeitmessung ausgesendeter Laserpulse und erzeugt dabei eine dichte Punktwolke der Umgebung. Die Lokalisierung erfolgt über Scan-Matching, bei dem die aktuell aufgenommene Punktwolke mit der gespeicherten Kartenpunktwolke zur Deckung gebracht wird, um daraus die aktuelle Pose zu bestimmen. Da die Tiefenmessung direkt und metrikgenau erfolgt, ist LiDAR-SLAM im Vergleich zu monokularem vSLAM nicht auf eine Skaleninitialisierung angewiesen und liefert in der Regel robustere Ergebnisse unter wechselnden Lichtverhältnissen [43]. Gegenüber kamerabasierten Verfahren sind LiDAR-Systeme jedoch deutlich teurer, schwerer und energieintensiver, was ihren Einsatz auf kleinen UAVs einschränkt.

3.2 Fiducial-Marker-Systeme

Im Folgenden wird ein Überblick über die am weitesten verbreiteten Fiducial-Marker-Systeme gegeben. Fiducial-Systeme wurden entwickelt, um in wenig strukturierten Umgebungen eine visuelle Lokalisierung gewährleisten zu können. Fiducial-Marker stellen ein gut unterscheidbares Muster mit starken visuellen Eigenschaften dar. Sie verfügen in den meisten Fällen auch über eine spezifische Kodierung als Sicherheitsmechanismus gegen Fehlerkennung.[44] Im Folgenden werden die in Abbildung 3.3 dargestellten gängigen Fiducial-Marker vorgestellt.

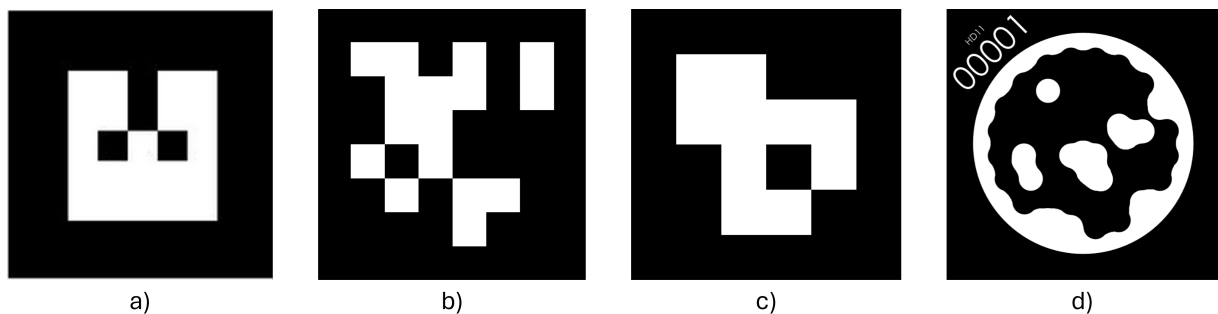


Abbildung 3.3: Übersicht gängiger Fiducial-Marker: a) ARTAG; b) AprilTag; c) Aruco d) Stag [45], [46]

3.2.1 ARTAG Marker

ARTAG wurde 2005 von Mark Fiala als Weiterentwicklung von ARToolKit vorgestellt.[47] Es verwendet digitale Kodierungsverfahren, wodurch ARTAG im Vergleich zu früheren Systemen niedrigere und quantifizierbare Fehlerraten aufweist.[47] Ein ARTAG besteht aus einem quadratischen Rand, der ein binäres Raster aus schwarzen und weißen Zellen umschließt, welches eine eindeutige ID kodiert.[47] Es lassen sich 2048 unterschiedliche IDs erzeugen, wobei 10 Bits für die Marker-ID und 26 Bits zur Redundanz genutzt werden.[47] ARTAG verwendet keine einheitliche minimale Hammingdistanz zwischen den Codewörtern.[47] Aus einer Aufnahme des Markers lässt sich eine 6D-Pose bestimmen.[47] ARTAG ist das einzige System, welches nicht vollständig Open-Source ist.

3.2.2 AprilTag Marker

AprilTag wurde 2011 von Edwin Olson an der University of Michigan vorgestellt.[48] AprilTag stellt eine schnellere und robustere Neuimplementierung des ARTag-Ansatzes dar.[48] Ein AprilTag besteht aus einem schwarzen quadratischen Rand, der ein binäres Raster aus schwarzen und weißen Zellen umschließt, welches eine eindeutige ID kodiert.[48] Das System ermöglicht die vollständige Bestimmung der sechs Freiheitsgrade der Pose aus einem einzelnen Bild und ist robust gegenüber Verdeckung, Verzerrung und Linsenverzeichnung.[48] Marker sind in verschiedene Familien unterteilt; das Namensschema TagNNhM beschreibt dabei NN als die Gesamtzahl der Datenbits und hM als die minimale Hammingdistanz.[48] Die Familie tag36h11 verwendet ein 6×6 -Raster mit einer minimalen Hammingdistanz von 11, die Familie tag16h5 ein 4×4 -Raster mit einer minimalen Hammingdistanz von 5.[48] tag16h5 lässt sich aufgrund der größeren Pixelgröße prinzipiell aus größerer Entfernung erkennen, jedoch mit höherer Unsicherheit.[48] In dieser Arbeit wird die Familie tag36h11 verwendet.[48] Mit AprilTag 2 wurde 2016 eine vollständig überarbeitete Detektion vorgestellt, die höhere Detektionsraten bei geringerem Rechenaufwand erreicht.[49]

3.2.3 ArUco Marker

ArUco wurde 2014 von Garrido-Jurado et al. vorgestellt.[50] Ein ArUco-Marker ist ähnlich wie ARTag und AprilTag aufgebaut. Er ist ebenfalls quadratisch und besitzt einen breiten Rand, in dessen Innerem sich eine Binärmatrix befindet, welche die Identifikationsnummer des Markers bestimmt.[50] Das System bietet einen Algorithmus zur automatischen Generierung konfigurierbarer Marker.[50] ArUco-Marker sind ebenfalls in Familien eingeteilt; das Namensschema $\text{DICT_N} \times \text{N_M}$ beschreibt dabei $\text{N} \times \text{N}$ als die Rastergröße der Datenbits und M als die maximale Anzahl der Marker im Dictionary.[50] Je kleiner das Dictionary, desto höher die Hammingdistanz zwischen den Markern.[50] ArUco ist in die Bibliothek OpenCV integriert.[50]

3.2.4 STag Marker

STag wurde 2019 von Benligiray et al. vorgestellt.[51] Im Gegensatz zu den rein quadratischen Systemen verwendet STag eine kreisförmige Randstruktur, die eine stabilere Posenschätzung ermöglicht.[51] Auch STags lassen sich in Familien unterteilen. So gibt es beispielsweise STag HD11, HD13 oder HD23, wobei HD für Hammingdistanz steht.[51] Je höher die Hammingdistanz, desto kleiner ist die Anzahl an möglichen Tags.[51]

3.2.5 Detektion von Fiducial-Markern

Alle vorgestellten Fiducial-Marker-Systeme stellen Bibliotheken zur Detektion bereit. Die Detektion eines Fiducial-Markers gliedert sich in zwei Hauptkomponenten: den eigentlichen Detektor, der die Position möglicher Marker im Bild schätzt, und das Kodierungssystem, das die ausgelesene ID auf Gültigkeit prüft.[48] Im Folgenden wird der Ablauf am Beispiel von AprilTag beschrieben.

Der Detektor sucht im Bild nach viereckigen, dunklen Flächen mit hellem Rand — den sogenannten Quads.[48] Dazu analysiert er, wo im Bild starke Helligkeitsübergänge auftreten, und fasst Bereiche mit ähnlichen Übergängen zu geraden Linien zusammen.[48] Da das Verfahren auf diesen

Helligkeitsunterschieden basiert, funktioniert es auch bei wechselnden Lichtverhältnissen zuverlässig.[48]

Aus je vier passenden Linien wird ein Quad gebildet; kleine Lücken oder Verdeckungen werden dabei toleriert.[48] Die genauen Eckpunkte des Quads ergeben sich aus den Schnittpunkten der Linien.[48]

Für jeden gefundenen Quad wird anschließend berechnet, wie das Bild des Markers entzerrt werden muss, um ihn frontal und perspektivisch unverzerrt darzustellen.[48] Aus dieser Entzerrung lassen sich zusammen mit der Brennweite der Kamera und der bekannten physischen Markergröße Position und Orientierung des Markers im Raum ableiten.[48] Abschließend werden die Datenbits des Markers ausgelesen und gegen das bekannte Dictionary geprüft – erst dieser Schritt bestätigt, dass es sich tatsächlich um einen gültigen Marker handelt, und reduziert Falschdetektionen.[48]

4 Systembeschreibung

In diesem Kapitel wird die Testumgebung beschrieben, sowie auf die Softwarearchitekturen eingegangen. Außerdem wird die Systempipeline erläutert, die zur Entwicklung der Echtzeiterkennung verwendet wurde. Es werden die verschiedenen Konzepte vorgestellt, die zur Auswahl eines Echtzeitmodells geführt haben.

4.1 Anforderungen an das Gesamtsystem

In diesem Teilkapitel werden die Anforderungen an das Gesamtsystem erläutert. Das Ziel der Arbeit ist es, ein UAV in einer GPS-freien Innenraumumgebung zu lokalisieren und diese Positionsschätzung als externe Referenz in die Flugsteuerung einzuspeisen. Das System soll dabei so ausgelegt sein, dass das UAV ohne Rückgriff auf ein GPS eine stabile Pose im Raum bestimmen kann. Anstelle einer satellitengestützten Positionsbestimmung wird die Pose des UAV aus einer externen, kamerabasierten Lokalisierung abgeleitet.

DZur Lokalisierung soll eine Karte der Versuchsumgebung genutzt werden, anhand derer die Position des UAV im Weltkoordinatensystem bestimmt werden kann. Die Karte soll erweiterbar sein, sodass der nutzbare Flugbereich bei Bedarf räumlich vergrößert werden kann.

Das System soll kostengünstig aufgebaut sein. Aus diesem Grund sollen anstelle spezialisierter Sensorik handelsübliche Kameras zur Lokalisierung eingesetzt werden. Die verwendete Software soll quelloffen und frei verfügbar sein.

Als Softwaregrundlage soll ROS2 eingesetzt werden, über das die einzelnen Verarbeitungsschritte miteinander verbunden werden. Die Flugsteuerung soll durch PX4 übernommen werden, der die extern berechnete Positionsschätzung verarbeitet.

Schließlich soll das System sicher betrieben werden können. Da es sich um ein fliegendes System handelt, muss sichergestellt sein, dass das UAV bei einem Ausfall oder einer fehlerhaften Positionsschätzung in einen sicheren Zustand übergehen kann.

Die Hardwarekomponenten, wie das UAV Holybro X650, der Flugcontroller Pixhawk 6X und der Companion-Computer Jetson Orin Nano, sind zur Verfügung gestellt und werden nicht im Rahmen dieser Arbeit entwickelt.

Die Ansteuerung der Kameras, die Lokalisierung des UAV, die Kartierung der Versuchsumgebung sowie die Anbindung der Positionsschätzung an die Flugsteuerung sind jedoch Teil der Arbeit und werden in dieser entwickelt.

4.2 Hardwareaufbau

Im Folgenden werden die einzelnen Hardwarekomponenten des Gesamtsystems sowie deren elektrischer Anschluss beschrieben. Das System besteht aus der Flugplattform, dem Companion-Computer, der die rechenintensive Bildverarbeitung übernimmt, und der Sensorik, die zur Lokalisierung eingesetzt wird.

Flugplattform

Als Flugplattform dient der bereits in den Grundlagen vorgestellte Holybro X650, der in der mit Motoren und Reglern vorkonfektionierten ARF-Variante (Almost Ready to Fly) verwendet wird. Das Gesamtgewicht des Kits beträgt ohne Akku rund 2000 g, die maximale Nutzlast ohne Akku liegt bei etwa 4,3 kg.[3]

Companion-Computer

Als Companion-Computer wird ein reComputer Mini J3011 von Seeed Studio eingesetzt. Dieser ist mit einem NVIDIA Jetson Orin Nano 8 GB Modul ausgestattet und erreicht eine KI-Rechenleistung von bis zu 40 Tera Operations Per Second (TOPS).[52] Der Companion-Computer übernimmt die Ansteuerung der Kameras, die Bildverarbeitung sowie die Berechnung der Positionsschätzung.

Das Modul wird ab Werk mit NVIDIA JetPack ausgeliefert, das auf einer Ubuntu-Distribution basiert. Für den Einsatz in dieser Arbeit wurde das System mit JetPack 6.2.1 neu geflasht, das auf Ubuntu 22.04.5 LTS und L4T R36.4.3 basiert, um die benötigte ROS2- und Bildverarbeitungssoftware betreiben zu können.

Flugcontroller

Die Flugsteuerung übernimmt der Flugcontroller Pixhawk 6X (FMU-V6X), auf dem der Autopilot PX4 ausgeführt wird. In dieser Arbeit wird PX4 in der Version 1.16.0 betrieben, die auf dem Echtzeitbetriebssystem NuttX v11.0.0 aufsetzt.

Kamerasysteme

Zur Lokalisierung stehen mehrere handelsübliche Universal Serial Bus (USB)-Kameras zur Verfügung, aus denen je nach Aufbau ausgewählt werden kann:

- eine USB-Kamera mit 720p-Auflösung `ali_camera`,
- eine Logitech C920,
- eine Logitech C922.

Die Kameras werden über USB an den Companion-Computer angeschlossen.

Zusätzlich steht eine Stereolabs ZED Mini zur Verfügung. Im Gegensatz zu den zuvor genannten Mono-Kameras handelt es sich um eine Stereokamera und steht somit für einen Visual-SLAM zur Verfügung. Sie verfügt zudem über eine integrierte IMU.

Fernsteuerung

Zur manuellen Steuerung des UAV sowie als Sicherheitsoption dient eine Radiolink AT9S Pro Fernsteuerung mit zugehörigem R9DS-Empfänger. Der Empfänger ist mit dem Flugcontroller verbunden und ermöglicht es, jederzeit manuell in den Flug einzugreifen.

Elektrischer Anschluss der Hardware

Die Energieversorgung des Systems erfolgt über einen LiPo-Akku mit einer Kapazität von 5400 mAh und einer Nennspannung von 14,8 V (4S).[53] Der Akku ist über ein Holybro PM02 V3 Powermodul angebunden.[54] Das Powermodul unterstützt Akkus von 2S bis 12S, stellt dem Flugcontroller eine geregelte Spannung von 5,2 V bereit und liefert zusätzlich Messwerte zu Batteriespannung und Stromaufnahme.[54] Die Verteilung der Motorströme erfolgt über ein Holybro Mini-Power-Distribution-Board (PDB) mit XT30- und XT60-Anschlüssen.

Die Kommunikation zwischen dem Flugcontroller und der Bodenstation erfolgt über ein Holybro Telemetrie-Set (433 MHz). Als Bodenstation dient ein Laptop mit Ubuntu 24.04.4 LTS, auf dem die Bodenstationssoftware QGroundControl betrieben wird. Zur satellitengestützten Positionsbestimmung ist ein Holybro M9N GPS-Modul vorhanden; dieses wird in der vorliegenden Arbeit jedoch nicht zur Lokalisierung verwendet. Die Verbindung zwischen dem Flugcontroller und dem Companion-Computer wird über eine serielle UART-Schnittstelle hergestellt, über die die

extern berechnete Positionsschätzung an den PX4 übermittelt wird. Für die drahtlose Anbindung des Companion-Computers ist dieser mit einer M.2-WLAN-Karte (Wi-Fi 6) nachgerüstet.

Tabelle 4.1 gibt eine Übersicht aller eingesetzten Komponenten mit Bezeichnung, Gewicht und Betriebssystem.

Komponente	Bezeichnung	Gewicht	Betriebssystem
Flugplattform	Holybro X650	2000 g	–
Companion-Computer	reComputer Mini J3011	–	JetPack 6.2 (Ubuntu 22.04)
Flugcontroller	Pixhawk 6X	–	PX4 v1.16.1 (NuttX)
Kamera 1	USB-Kamera 720p	–	–
Kamera 2	Logitech C920	–	–
Kamera 3	Logitech C922	–	–
Kamera 4	Stereolabs ZED Mini	62,9 g	–
Fernsteuerung	Radiolink AT9S Pro / R9DS	–	–
Akku	Pichler RED POWER XT 5400 (4S)	–	–
Powermodul	Holybro PM02 V3	20 g	–
Stromverteilung	Holybro Mini-PDB	–	–
Telemetrie	Holybro Telemetrie-Set 433 MHz	–	–
GPS-Modul	Holybro M9N GPS	–	–
WLAN-Karte	M.2 Wi-Fi 6 Adapter	–	–
Bodenstation	Laptop / QGroundControl	–	Ubuntu 24.04.4 LTS

Tabelle 4.1: Übersicht der eingesetzten Hardwarekomponenten

In Abbildung 4.1 ist der Hardwareaufbau des Gesamtsystems als Blockschaltbild mit Anschlussverdrahtung dargestellt. Schwarze Linien kennzeichnen dabei Daten- und Signalverbindungen, rote Linien die Energieversorgung.

Die drei Kameras können über USB mit dem Companion-Computer verbunden. Der Companion-Computer kann sowohl über einen USB-zu-Ethernet-Adapter an die Ethernet-Schnittstelle des Flugcontrollers angebunden werden, als auch direkt via USB. Am Flugcontroller sind weiterhin das Telemetriemodul, das GPS-Modul sowie der Empfänger der Fernsteuerung angeschlossen. Die Ansteuerung der vier Motoren erfolgt über die PWM-Ausgänge des Flugcontrollers.

Die Energieversorgung erfolgt ausgehend vom LiPo-Akku. Der Akku ist über einen XT90-Stecker mit dem Powermodul verbunden, das die Versorgungsspannung an den Flugcontroller und das Verteilerboard weitergibt. Vom Verteilerboard wird die Energie über XT30-Anschlüsse an die vier Motoren verteilt. Parallel dazu wird der Companion-Computer über einen eigenen XT30-Anschluss mit Strom versorgt.

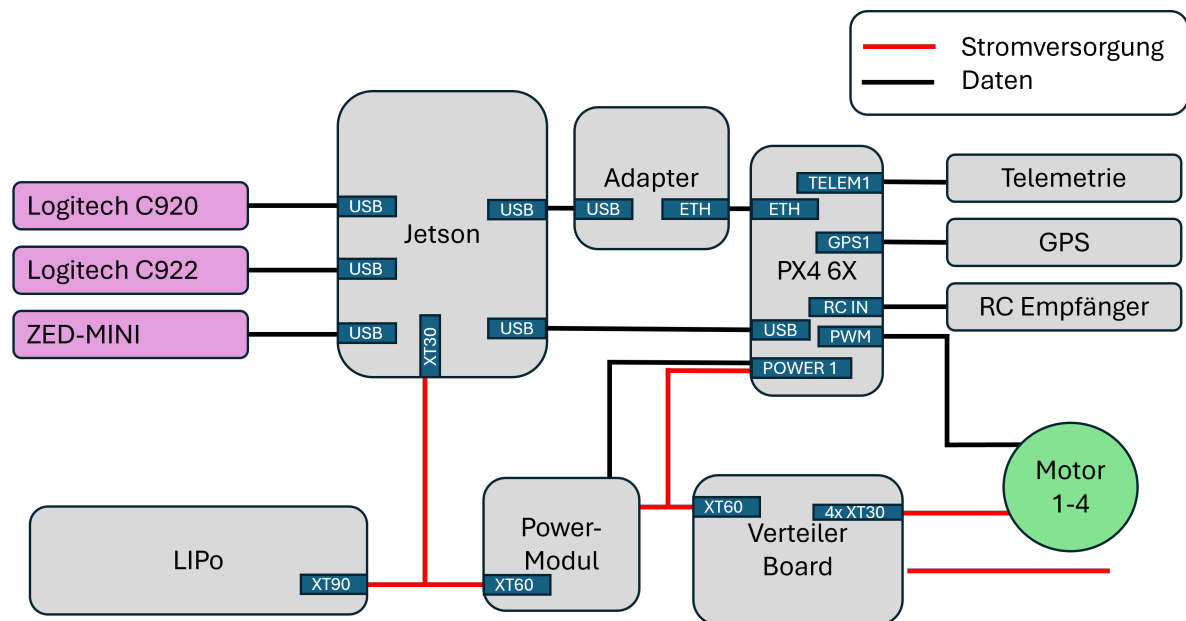


Abbildung 4.1: Hardware-Blockschaltbild mit Anschlussverdrahtung

4.3 Softwarearchitektur Kartierung und Positionierung nochmal drüber schauen!!! ich nehme ja auch das thinkpad!!!

In diesem Abschnitt wird die Softwarearchitektur des Gesamtsystems beschrieben. Die Verarbeitung gliedert sich in zwei Stränge: die Lokalisierung, die im Flug die Pose des Fluggeräts bestimmt und die Kartierung, die vorab die dafür benötigte Karte erzeugt. Beide Stränge laufen auf dem Companion-Computer und teilen sich die vorgelagerten Schritte Bildaufnahme, Entzerrung und Bildverarbeitung. Die Flugsteuerung läuft getrennt davon auf dem Flugcontroller. Der Aufbau ist in Abbildung 4.3 dargestellt.

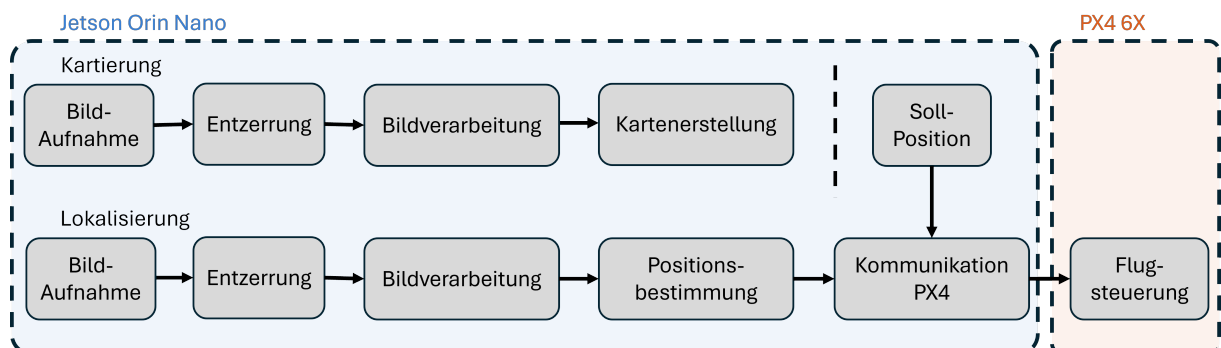


Abbildung 4.2: Softwarearchitektur mit den Strängen zur Lokalisierung und zur Kartierung

Die Bildaufnahme bildet die Schnittstelle zu den am Fluggerät montierten Kameras. Über dieses Modul werden die Kamerabilder aufgenommen und für die weitere bereitgestellt.

Die Entzerrung korrigiert die durch das Kameraobjektiv verursachte Verzeichnung anhand der zuvor bestimmten Kalibrierparameter. Ohne diese Korrektur würden die Bildkoordinaten der erkannten Merkmale systematisch verfälscht und die daraus berechnete Pose verzerrt. Die Grundlagen des Kameramodells und der Kalibrierung werden in Abschnitt 5.2.3 behandelt.

Die Bildverarbeitung detektiert und extrahiert die markanten Bildmerkmale aus entzerrten Kamerastream. Für jedes Merkmal liefert die Bildverarbeitung die relative Position zwischen Kamera und Merkmal.

Die Kartierung erzeugt die Karte, auf die sich die Positionsbestimmung stützt. Sie nutzt dieselbe Aufnahme- und Verarbeitungskette wie die Lokalisierung, bestimmt daraus aber nicht die UAV-Pose, sondern die Lage der Merkmale zueinander. Das Ergebnis ist eine konsistente Karte aller Merkmale, die einmalig erzeugt und anschließend im Flug verwendet wird. Die Umsetzung wird in Kapitel 5 beschrieben.

Die Positionsbestimmung bildet den Kern der Lokalisierung. Sie verknüpft die erkannten Merkmale mit der vorab erstellten Karte und berechnet daraus die Pose der Kamera im Kartenkoordinatensystem. Anschließend wird die Kamerapose zum Roboterkoordinatensystem transformiert. Die Umsetzung der Lokalisierung wird in Kapitel 6 erläutert.

Die Kommunikation überführt die berechnete Pose vom Karten- in das Koordinatensystem der Flugsteuerung und versieht sie mit einem dazu konsistenten Zeitstempel. Für die Übergabe an den Flugcontroller wurden zwei Implementierungen umgesetzt. Die Umsetzung wird in Abschnitt 6.7 beschrieben.

Die Flugsteuerung läuft auf dem PX4-Flugcontroller und regelt das Fluggerät. Sie fusioniert die eingespeiste Pose im Zustandsschätzer mit den Daten der IMU und überbrückt damit kurze Phasen, in denen keine Position bestimmt werden kann. Der Aufbau der Zustandsschätzung wird in Abschnitt 6.8 beschrieben.

Das Senden der Sollpositionen läuft parallel zur Positionsbestimmung des UAV. Die Funktionsweise ist in Abschnitt ?? beschrieben.

4.4 Versuchsumgebung

Die Versuche werden im AerodrOhm der Technischen Hochschule Nürnberg durchgeführt, einer Indoor-Flughalle am Ohm Innovation Center.[55] Die Halle bietet eine Grundfläche von 280 m^2 bei einer nutzbaren Flughöhe von bis zu 12 m.



Abbildung 4.3: Flughalle AerodrOhm am Ohm Innovation Center (OIC) der Technischen Hochschule Nürnberg [55]

Der Hallenboden ist mit einem spiegelnden Belag ausgestattet. Zwei der vier Hallenwände bestehen aus unverputztem Ziegelmauerwerk und weisen daher eine strukturierte Oberfläche auf. Die übrigen zwei Wände sind mit großflächigen Fenstern versehen, deren Jalousien je nach Zustand geöffnet oder geschlossen sein können. Dies hat unmittelbaren Einfluss auf die Lichtverhältnisse in der Halle: Bei geöffneten Jalousien kann einfallender Tageslicht zu Gegenlicht führen, sobald die Kamera in Richtung der Fensterflächen ausgerichtet ist.

5 Implementierung der Kartierung

In diesem Kapitel werden die Schritte beschrieben, die zur Implementierung der Kartierung notwendig sind. Hierbei werden Festlegungen getroffen, die auch direkten Einfluss auf die in Kapitel 6 beschriebene Lokalisierung haben. Im weiteren Verlauf der Arbeit wird auf diese Abhängigkeiten hingewiesen.

5.1 Auswahl der Algorithmen

Da die Karte als spätere Referenz für die Lokalisierung des UAV dient, überträgt sich ihre Genauigkeit unmittelbar auf den Lokalisierungsfehler. Um eine möglichst genaue Karte zu erhalten und den relativen Fehler der Karte sowie der Lokalisierung gering zu halten, kommen daher ausschließlich optimierungsbasierte SLAM-Methoden zum Einsatz.

5.1.1 Auswahl der Bewertungskriterien SLAM-Varianten

Die Bewertung erfolgt anhand von sieben Kriterien, die sich aus den Anforderungen dieser Arbeit ableiten. **Kostengünstigkeit** und **Leichtbau** beziehen sich auf die benötigte Sensorhardware. Beide sind relevant, da die Lokalisierung auf einem UAV mit begrenzter Nutzlast und mit geringem monetären Aufwand umgesetzt werden soll. Der **geringe Vorbereitungsaufwand** bewertet den einmaligen Aufwand, das System einsatzbereit zu machen, etwa das Anbringen von Markern. Die **Umgebungskompatibilität** beschreibt, wie zuverlässig ein Verfahren unter den Bedingungen der Versuchsumgebung arbeitet, insbesondere bei strukturarmen Flächen und dem spiegelnden Hallenboden und Gegenlicht. Die **Driftfreiheit** bewertet, ob ein Verfahren über die Zeit einen absoluten Positionsbezug behält oder einen akkumulierenden Fehler aufweist. Die **ROS2-Verfügbarkeit** berücksichtigt, ob für das Verfahren einsatzfähige Implementierungen im verwendeten Framework vorliegen. Der **Rechenaufwand** bewertet die Belastung des Companion-Computers, da die Verarbeitung onboard erfolgt; eine höhere Bewertung entspricht einem geringeren Rechenaufwand.

5.1.2 Gewichtung durch den paarweisen Vergleich der SLAM-Varianten

Im paarweisen Vergleich (Tabelle 5.1) wird jedes Kriterium einzeln gegen jedes andere abgewogen. Eine 1 bedeutet, dass das Kriterium der Zeile gegenüber dem der Spalte als wichtiger eingestuft wird. Aus der Zeilensumme ergibt sich die relative Gewichtung jedes Kriteriums.

Die höchste Gewichtung erhalten die **Umgebungskompatibilität** und die **Driftfreiheit** mit jeweils 23,81 %. Diese Priorisierung ergibt sich direkt aus dem Ziel der Arbeit, ein Lokalisierungssystem für einen Multikopter im Innenraum zu implementieren. Ein Verfahren, das in der Versuchsumgebung nicht funktioniert oder stark driftbehaftet ist, wäre für den vorgesehenen Einsatz unabhängig von seinen sonstigen Eigenschaften ungeeignet.

Die **ROS2-Verfügbarkeit** und der **Rechenaufwand** folgen mit jeweils 19,05 %. Beide sind für die praktische Umsetzbarkeit auf der verwendeten Plattform entscheidend, ordnen sich den Anforderungen an die Lokalisierungsqualität jedoch unter. Die hardwarebezogenen Kriterien **Kostengünstigkeit**, **Leichtbau** und **geringer Vorbereitungsaufwand** erhalten mit jeweils 4,76 % das geringste Gewicht.

Kostengünstigkeit und geringer Vorbereitungsaufwand sind zwar wünschenswert, für die grundsätzliche Eignung des Verfahrens jedoch von untergeordneter Bedeutung. Auch der Leichtbau spielt eine geringere Rolle, da die Nutzlast des verwendeten UAVs ausreichend ist, um alle betrachteten Hardwarevarianten zu tragen.

Paarweiser Vergleich

als wichtiger	Kostengünstigkeit	Leichtbau	Geringer Vorbereitungsaufwand	Umgebungs- kompatibilität	Driftfreiheit	ROS2-Verfügbarkeit	Rechenaufwand	Summe	%
Kostengünstigkeit		0	1	0	0	0	0	1	4,76%
Leichtbau	1		0	0	0	0	0	1	4,76%
Geringer Vorbereitungsaufwand	0	1		0	0	0	0	1	4,76%
Umgebungs-kompatibilität	1	1	1		1	1	0	5	23,81%
Driftfreiheit	1	1	1	0		1	1	5	23,81%
ROS2-Verfügbarkeit	1	1	1	0	0		1	4	19,05%
Rechenaufwand	1	1	1	1	0	0		4	19,05%
Prüfsumme									100,00%

Bewertung 1 bedeutet im Beispiel Kriterium 1 ist wichtiger als Kriterium 2
Bewertung 1 bedeutet im Beispiel Kriterium 1 ist wichtiger als Kriterium 2

Abbildung 5.1: Paarweiser Vergleich Bewertungskriterien der SLAM-Varianten

5.1.3 Nutzwertanalyse SLAM-Varianten

Zur Auswahl der SLAM-Variante werden die in Abschnitt 3.1.3.3 vorgestellten Verfahren vSLAM, TagSLAM und LiDAR-SLAM anhand einer Nutzwertanalyse gegenübergestellt.

In der Nutzwertanalyse (Tabelle 5.2) wird jede Variante in jedem Kriterium auf einer Skala von 0 (Kriterium nicht erfüllt) bis 5 (Kriterium vollständig erfüllt) bewertet. Der gewichtete Wert ergibt sich aus dem Produkt von Bewertung und Kriteriengewicht, die Summe dieser Werte bildet den Gesamtnutzwert. Alle Verfahren werden dabei im Lokalisierungsmodus gegen eine zuvor erstellte Karte betrachtet.

In der **Umgebungs-kompatibilität** erreichen TagSLAM und LiDAR-SLAM mit jeweils 4 die höchste Bewertung. TagSLAM arbeitet unabhängig von der Umgebungstextur und wird daher von strukturalarmen Flächen nicht beeinträchtigt, während LiDAR-SLAM unabhängig von der Beleuchtung misst. VSLAM wird mit 3 am niedrigsten bewertet, da es auf eine texturreiche Umgebung angewiesen ist, die in der Versuchsumgebung nicht zuverlässig vorliegt.

In der **Driftfreiheit** erreicht TagSLAM mit 5 den höchsten Wert, da jede Markerdetektion eine direkte und absolute Posenmessung gegen die Karte liefert. LiDAR-SLAM bindet die Pose über das Registrieren der Punktwolke ebenfalls an die Karte, erhält mit 4 aber eine geringere Bewertung, da das Scan-Matching in strukturalarmen Bereichen mehrdeutig werden kann [56]. VSLAM wird mit 3 bewertet, da die Merkmalswiedererkennung in der strukturalarmen Versuchsumgebung weniger zuverlässig ist.

lässig gelingt und das Verfahren dadurch häufiger auf die inhärent driftbehaftete visuelle Odometrie angewiesen ist [57].

Beim **Rechenaufwand** schneidet TagSLAM mit 4 am besten ab, da durch die bekannte Marker-geometrie die aufwendige Merkmalsbeschreibung entfällt. LiDAR-SLAM wird hier mit 2 am niedrigsten bewertet, da die Verarbeitung dichter Punktwolken rechenintensiv ist. Diese Bewertung ist als qualitative Tendenz zu verstehen, da der tatsächliche Rechenaufwand stark von der jeweiligen Algorithmenimplementierung abhängt. Die **ROS2-Verfügbarkeit** ist für alle drei Verfahren mit 4 gleich bewertet, da für jedes einsatzfähige Implementierungen vorliegen, die gegebenenfalls noch angepasst werden müssen.

LiDAR-SLAM wird bei **Kostengünstigkeit** und **Leichtbau** mit jeweils 2 am niedrigsten bewertet, da ein kompakter 3D-LiDAR-Sensor wie der Livox Mid-360S mit 589 Euro [58] deutlich teurer und schwerer ist als eine vergleichbare Kamera. Für kamerabasierte Verfahren ist hingegen bereits eine Logitech C922 für unter 100 Euro im Handel erhältlich [59]. Ein für vSLAM ausgelegtes Stereomodul wie die ZED Mini liegt mit ab 399 USD [60] zwar höher, bleibt aber deutlich unter dem Preisniveau eines LiDAR-Sensors. TagSLAM wird beim **geringen Vorbereitungsaufwand** mit 3 bewertet, da die Marker einmalig angebracht werden müssen, während VSLAM und LiDAR-SLAM mit 5 keine Umgebungsvorbereitung erfordern.

In Summe erreicht TagSLAM mit 4,24 den höchsten Gesamtnutzwert, gefolgt von LiDAR-SLAM mit 3,48 und VSLAM mit 3,38. Ausschlaggebend ist die hohe Bewertung in den beiden am stärksten gewichteten Kriterien, der Umgebungskompatibilität und der Driftfreiheit. Der höhere Vorbereitungsaufwand als einziger nennenswerter Nachteil von TagSLAM fällt aufgrund der geringen Gewichtung dieses Kriteriums kaum ins Gewicht. Aus diesen Gründen wird die Kartierung und Lokalisierung in dieser Arbeit auf Basis von TagSLAM umgesetzt.

Nutzwertanalyse

	Gewichtung	VSLAM		TagSLAM		LiDAR-SLAM	
		Bewertung	Wert	Bewertung	Wert	Bewertung	Wert
Kostengünstigkeit	4,76%	3	0,14	5	0,24	2	0,10
Leichtbau	4,76%	5	0,24	4	0,19	2	0,10
Geringer Vorbereitungsaufwand	4,76%	5	0,24	3	0,14	5	0,24
Umgebungs-kompatibilität	23,81%	3	0,71	4	0,95	4	0,95
Driftfreiheit	23,81%	3	0,71	5	1,19	4	0,95
ROS2-Verfügbarkeit	19,05%	4	0,76	4	0,76	4	0,76
Rechenaufwand	19,05%	3	0,57	4	0,76	2	0,38
Summe			3,38		4,24		3,48

Bewertungszahl von 0 - 5

Bewertungszahl 0 entspricht Alternative erfüllt das Kriterium nicht

Bewertungszahl 5 entspricht Alternative erfüllt das Kriterium vollständig

Abbildung 5.2: Nutzwertanalyse der SLAM-Varianten

5.1.4 Auswahl der Fiducial-Marker

Als Fiducial-Marker-System kommen in dieser Arbeit ausschließlich AprilTags der Familie `tag36h11` zum Einsatz. Die Wahl ergibt sich aus drei zusammenwirkenden Gründen.

Kompatibilität mit TagSLAM. TagSLAM von Bernd Pfrommer nutzt intern den AprilTag-Detektor und unterstützt ausschließlich AprilTag-Familien [42]. Ein Wechsel zu einem anderen Marker-System würde die Kompatibilität mit TagSLAM aufheben.

Hardwarebeschleunigte Detektion unter Isaac ROS. Für den Jetson Orin Nano steht mit dem Paket `isaac_ros_apriltag` eine Graphics Processing Unit (GPU)-beschleunigte Implementierung zur Detektion von AprilTag-Markern zur Verfügung. Da AprilTag im Vergleich zu ArUco den höchsten Rechenaufwand bei einer CPU-basierten Implementierung aufweist [44], ist die Hardwarebeschleunigung für den echtzeitfähigen Betrieb auf dem Companion-Computer besonders relevant. Eine vergleichbare Beschleunigung für ArUco-Marker steht im Isaac ROS-Framework nicht zur Verfügung.

Wahl der Familie `tag36h11`. Innerhalb der AprilTag-Familien bietet `tag36h11` den besten Kompromiss aus Detektionsleistung und Robustheit gegenüber Falschdetektionen [61]. Die minimale Hammingdistanz von 11 stellt sicher, dass eine große Bitabweichung erforderlich ist, um einen ungültigen Marker als gültig zu akzeptieren, was die Verwechslungsrate gegenüber Familien mit geringerer Hammingdistanz wie `tag16h5` deutlich reduziert [48].

Anforderungen an das Anbringen der Marker. Für eine präzise Posenschätzung müssen die Marker eben und ohne Verwölbung angebracht sein, da das Detektionsverfahren eine planare Mar-

kergeometrie voraussetzt [48]. Verformungen des Untergrunds oder des Druckträgers führen zu systematischen Fehlern in der berechneten Pose. Weiterhin ist ein ausreichend hoher Kontrast zwischen den schwarzen und weißen Zellen des Markers entscheidend, da die Detektion auf Helligkeitsübergängen basiert [44]. Glänzende Oberflächen sind zu vermeiden, da Reflexionen die Helligkeitsübergänge verfälschen und zu Detektionsfehlern führen können.

5.2 Ablauf der Kartierung

Für die Kartierung wurden in dieser Arbeit zwei Ansätze verfolgt. Zum einen die Live-Kartierung mittels Tag-SLAM während des Flugs und zum anderen die Offline-Kartierung durch Aufzeichnung von ROS2-Bags (siehe Kapitel 2.6.8).

Es hat sich gezeigt, dass die Live-Kartierung nur bei kleinen Karten zuverlässig funktioniert, da die Optimierung bei größeren Karten zu lange dauert. Es konnte keine Konfiguration gefunden werden, mit der TagSLAM die Live-Kartierung einer großen Karte in akzeptabler Zeit bewältigt. Aus diesem Grund wird die Karte vorab in einem dreistufigen Offline-Verfahren erstellt, das unter ROS2 Jazzy umgesetzt ist. Jede Stufe ist als eigenes Launch-File umgesetzt, die Stufen sind über ROS2-Bags miteinander verbunden.

In der ersten Stufe werden die Kamerabilder aufgenommen, entzerrt und gemeinsam mit den Kameraparametern in einem ROS2-Bag aufgezeichnet. In der zweiten Stufe werden die Marker offline aus den aufgezeichneten Bildern detektiert und die Detektionen zusammen mit den Bildern in einem zweiten Bag abgelegt. In der dritten Stufe werden aus diesen Detektionen mit TagSLAM die Lagen aller Marker zu einer konsistenten Karte optimiert.

Der Offline-Ablauf bietet zwei Vorteile. Da die aufgezeichneten Daten unabhängig von der Echtzeit verarbeitet werden, kann die Detektion mit reduzierter Abspielgeschwindigkeit erfolgen, sodass jedes Bild ausgewertet wird und keine Detektionen verloren gehen. Zudem ist der gesamte Vorgang reproduzierbar, da er beliebig oft aus denselben Eingangsdaten wiederholt werden kann. Die einzelnen Schritte werden in den folgenden Abschnitten beschrieben.

Die Kalibrierung sowie die Offline-Kartierung werden auf einem Lenovo ThinkPad mit einem Intel Core i5-1145G7 (11. Generation), 16 GB Arbeitsspeicher und integrierter Intel Iris Xe-Grafik unter Ubuntu 24.04 mit ROS2 Jazzy durchgeführt.

5.2.1 Kamera-Kalibrierung

Die Kamerakalibrierung ist die Voraussetzung für die in Abschnitt 3.2.5 beschriebene Posenbestimmung. Dort werden aus der Entzerrung des Markers zusammen mit der Brennweite der Kamera und der bekannten physischen Markergröße Position und Orientierung des Markers im Raum abgeleitet. Die intrinsischen Parameter der Kamera (Brennweite, Hauptpunkt und Verzeichnungskoeffizienten) müssen daher vor der Detektion bekannt sein. Reale Objektive weichen vom idealen Lochkamera-modell ab und erzeugen eine Linsenverzeichnung, durch die gerade Linien im Bild gekrümmt abgebildet werden. Ohne Kalibrierung würde diese Verzeichnung unmittelbar in die berechnete Markerpose eingehen und zu einem systematischen Fehler in der Karte sowie in der späteren Lokalisierung führen [62]. Die Kalibrierung erfolgt nach dem Verfahren von Zhang [62] unter Verwendung eines planaren Schachbrettmusters, das aus mehreren Blickwinkeln aufgenommen wird. Die Eckpunkte

des Schachbretts werden dabei automatisch detektiert und zur Schätzung der intrinsischen Parameter und der Verzeichnungskoeffizienten verwendet, wie es in der OpenCV-Bibliothek implementiert ist [11]. Die ermittelten Parameter gelten nur für die jeweils verwendete Kamera in der jeweils eingestellten Auflösung. Daher muss die Kalibrierung für jede Kamera, auch bei baugleichen Modellen, einzeln durchgeführt werden. Ebenso ist die Kalibrierung bei jeder Änderung der Auflösung erneut durchzuführen, da Brennweite und Hauptpunkt von der Auflösung abhängen.

Die Kalibrierung wird mit dem ROS2-Paket `camera_calibration` umgesetzt. Zunächst wird die Kamera über `gscam` gestartet:

Quellcode 5.1: Start des gscam-Knotens für die Kalibrierung

```
1 ros2 run gscam gscam_node --ros-args \
2   -p gscam_config:="v4l2src device=/dev/video4 do-timestamp=true ! \
3     image/jpeg,width=1920,height=1080,framerate=30/1 ! \
4     jpegdec ! videoconvert ! video/x-raw,format=RGB" \
5   -p frame_id:=camera_optical_frame \
6   -p camera_name:=camera
```

In einem zweiten Terminal wird der Kalibrierknoten gestartet:

Quellcode 5.2: Start des Kalibrierknotens

```
1 ros2 run camera_calibration cameracalibrator \
2   --size 9x6 \
3   --square 0.032 \
4   --no-service-check \
5   --ros-args \
6   --remap image:=/image_raw
```

Der Parameter `--size 9x6` gibt die Anzahl der inneren Eckpunkte des Schachbretts an, `--square 0.032` die Kantenlänge eines Schachbrettfelds in Metern. Aus mehreren Aufnahmen des Schachbretts in verschiedenen Lagen werden die Kameraparameter bestimmt.

Das Ergebnis der Kalibrierung sind die folgenden Parameter: Die Kameramatrix $\mathbf{K}$ ordnet den ermittelten Werten die intrinsischen Parameter zu:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1470,05 & 0 & 926,22 \\ 0 & 1463,81 & 506,44 \\ 0 & 0 & 1 \end{bmatrix}. \quad (5.1)$$

Der Verzeichnungsvektor $\mathbf{d}$ enthält die radialen und tangentialen Koeffizienten:

$$\mathbf{d} = [k_1 \ k_2 \ p_1 \ p_2 \ k_3] = [0,0178 \ -0,0436 \ -0,0015 \ -0,0044 \ 0]. \quad (5.2)$$

Die Rektifizierungsmatrix $\mathbf{R}$ entspricht bei einer Einzelkamera der Einheitsmatrix:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (5.3)$$

Die Projektionsmatrix $\mathbf{P}$ beschreibt die intrinsischen Parameter des entzerrten Bildes:

$$\mathbf{P} = \begin{bmatrix} f'_x & 0 & c'_x & 0 \\ 0 & f'_y & c'_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1467,68 & 0 & 916,64 & 0 \\ 0 & 1469,52 & 504,58 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}. \quad (5.4)$$

Die **Kameramatrix** enthält die intrinsischen Parameter und beschreibt, wie ein Punkt aus dem dreidimensionalen Raum auf die zweidimensionale Bildebene projiziert wird [11]. Auf der Hauptdiagonalen stehen die Brennweiten $f_x = 1470,05$ und $f_y = 1463,81$, angegeben in Pixeln. Die Brennweite bestimmt den Abbildungsmaßstab, also wie stark ein Objekt abhängig von seiner Entfernung verkleinert auf den Sensor abgebildet wird [11]. In der dritten Spalte steht der Hauptpunkt $(c_x, c_y) = (926,22; 506,44)$, also der Punkt auf dem Bildsensor, an dem die optische Achse senkrecht auftrifft [11]. Idealerweise läge dieser in der Bildmitte (bei 960; 540), die reale Lage weicht jedoch fertigungsbedingt geringfügig davon ab.

Der **Verzeichnungsvektor** beschreibt, wie stark das reale Objektiv vom idealen Lochkameramodell abweicht, in der Reihenfolge $(k_1, k_2, p_1, p_2, k_3)$.

Die radialen Koeffizienten k_1, k_2, k_3 erfassen die zur Bildmitte symmetrische Verzeichnung, die durch die Krümmung der Linse entsteht. Sie bewirkt, dass gerade Linien zum Bildrand hin zunehmend gekrümmt erscheinen, während die Bildmitte nahezu unverzerrt bleibt. k_1 beschreibt den dominanten Anteil dieser Verzeichnung, k_2 und k_3 korrigieren die verbleibende Abweichung bei stärkerer Krümmung am äußeren Bildrand.

Die tangentialen Koeffizienten p_1, p_2 erfassen eine Verschiebung, die entsteht, wenn die Linse nicht exakt parallel zur Sensorebene montiert ist. Dadurch wird das Bild leicht asymmetrisch verzerrt, da der Abstand zwischen Linse und Sensor über die Bildfläche nicht konstant ist.

Die **Rektifizierungsmatrix** dient der Ausrichtung mehrerer Kameras in einem Stereoaufbau [11]. Da hier eine einzelne Kamera verwendet wird, entspricht sie der Einheitsmatrix und bewirkt keine Drehung.

Die **Projektionsmatrix** beschreibt die intrinsischen Parameter, die für das bereits entzerrte Bild gelten [11]. Während die Kameramatrix die Abbildung des realen, verzeichneten Objektivs beschreibt, bezieht sich die Projektionsmatrix auf das Bild, nachdem die Verzeichnung rechnerisch entfernt wurde. Da das entzerrte Bild einem idealen Lochkameramodell entspricht, enthält die Projektionsmatrix keine Verzeichnungskoeffizienten mehr.

Ihre ersten drei Spalten enthalten erneut Brennweite und Hauptpunkt, die durch die Entzerrung leicht von den Werten der ursprünglichen Kameramatrix abweichen können. Die vierte Spalte ist bei einer Einzelkamera null und erhält nur im Stereoaufbau einen von null verschiedenen Eintrag, der die Verschiebung zur zweiten Kamera beschreibt. In der nachfolgenden Bildverarbeitung wird daher die Projektionsmatrix anstelle der Kameramatrix verwendet, da sie die tatsächliche Abbildung des entzerrten Bildes beschreibt.

5.2.2 Bildaufnahme Kartierung

Die Kartierung kann mit einer beliebigen Kamera durchgeführt werden, wichtig ist die in Abschnitt 5.2.1 beschriebene Kalibrierung für die verwendete Kamera durchzuführen. Die Implementierung der Kartierung wird am Beispiel der Logitech C922 im folgenden erläutert. Die Kartierung ist auch mit anderen Kameramodellen möglich, die Parameter müssen entsprechend der hardware angepasst werden.

Die Bildaufnahme erfolgt über einen gscam-Knoten, der in das Launch-File der ersten Stufe eingebunden ist. Listing 5.3 zeigt dessen Konfiguration.

Die Bildaufnahme erfolgt über einen gscam-Knoten, der in das Launch-File der ersten Stufe eingebunden ist. Listing 5.3 zeigt dessen Konfiguration.

Quellcode 5.3: Parameter des gscam-Knotens zur Bildaufnahme

```
1 'gscam_config': (
2     'v4l2src device=/dev/video4 do-timestamp=true ! '
3     'image/jpeg,width=1920,height=1080,framerate=15/1 ! '
4     'queue leaky=downstream max-size-buffers=1 ! '
5     'jpegdec ! '
6     'queue leaky=downstream max-size-buffers=1 ! '
7     'videoconvert ! '
8     'video/x-raw,format=RGB ! '
9     'queue leaky=downstream max-size-buffers=1'
10 ),
11 'camera_name': 'camera',
12 'camera_info_url': camera_info_url,
13 'frame_id': 'camera',
14 'use_gst_timestamps': True,
15 'sync_sink': False,
16 'preroll': False,
17 'image_encoding': 'rgb8',
18 'use_sensor_data_qos': False,
```

Die wichtigsten Parameter sind die Auflösung von 1920×1080 Pixeln bei einer Bildfrequenz von 15 frames per second (fps). Die Bildfrequenz ist bewusst niedrig gewählt, um die Dekodierung durch jpegdec auf der CPU nicht zu überfordern und eine stabile Bildrate zu erhalten. Über camera_info_url werden die zuvor ermittelten Kalibrierparameter geladen, die zusammen mit dem Bild auf den Topics /camera/image_raw und /camera/camera_info veröffentlicht werden. Im Anschluss an die Aufnahme werden die Bilder direkt entzerrt (siehe Abschnitt 5.2.3).

5.2.3 Entzerrung

Die Entzerrung wird über den Knoten rectify_node aus dem Paket image_proc umgesetzt. Anhand der Kalibrierparameter ordnet er jedem Pixel des Ausgabebildes die zugehörige Position im verzeichneten Eingangsbild zu und entfernt dadurch die Linsenverzeichnung. Das Ergebnis wird als Topic /camera/image_rect bereitgestellt.

Nach diesem Schritt ist das Bild bereits entzerrt. Seine intrinsischen Parameter entsprechen damit

der Projektionsmatrix aus Gleichung (5.4), und die Verzeichnungskoeffizienten sind null. Anschließend wird ein ROS2-Bag aufgezeichnet, der die für die nachfolgende Stufe benötigten Topics enthält:

- `/camera/image_rect` – das entzerrte Bild,
- `/camera/camera_info` – die zugehörigen Kameraparameter,
- `/tf_static` – die statischen Koordinatentransformationen.

5.2.4 Offline Detektion von AprilTags

Die Markererkennung wird offline auf dem in der ersten Stufe aufgezeichneten Bag mit den entzerrten Bildern ausgeführt. Die Detektion ist absichtlich von Aufnahme und Entzerrung entkoppelt, um die genutzte Hardware nicht zu überfordern. Es wurden auch Versuche durchgeführt, bei denen die Detektion Teil der ersten Bag-Aufzeichnung war. Dabei führte die zu hohe Rechenlast bei Echtzeit-Abspielgeschwindigkeit dazu, dass Frames verworfen wurden und die Detektion nicht mehr zuverlässig funktionierte. Bei leistungsfähigerer Hardware könnten beide Schritte gegebenenfalls gleichzeitig durchgeführt werden. Um der Hardware pro Bild mehr Zeit zur Berechnung zu geben, wird der Eingabsbag mit einer Abspielgeschwindigkeit von 0,3 abgespielt.

Für die Detektion wird der Knoten `sync_and_detect_node` aus dem Paket `TagSLAM` verwendet. Dieser benötigt zwei Konfigurationsdateien, auf diese wird in Abschnitt ?? nochmals genauer eingegangen. Die Datei `cameras.yaml` enthält die Intrinsics der Kamera. Da der Knoten auf dem bereits entzerrten Bild `/camera/image_rect` arbeitet, werden hier die Werte der Projektionsmatrix aus Abschnitt ?? eingetragen und die Verzeichnungskoeffizienten auf null gesetzt. Neben den Intrinsics legt die Datei die Auflösung, die verwendeten Topics (`image_topic`, `camera_info_topic`, `tag_topic`) sowie den Referenzframe der Kamera (`rig_body`) fest. Aus der `TagSLAM`-Konfiguration `tagslam.yaml` liest der Knoten die Liste der bekannten Marker mit ihren Identifikationsnummern und physischen Größen aus. Diese Information macht es möglich, Karten mit verschiedenen großen Tags aufzunehmen. Der Knoten synchronisiert die Topics `/camera/image_rect` und `/camera/camera_info_rect` anhand ihrer Zeitstempel und detektiert auf den zugehörigen Bildern die AprilTag-Marker. Für jeden erkannten Marker werden Identifikationsnummer und Bildkoordinaten der vier Eckpunkte bestimmt und als Topic `/tagslam/tag_detections` ausgegeben.

Die Detektionen werden zusammen mit den Bildern, den Kameraparametern, den statischen Transformationen und der Zeit aus dem Eingangs-Bag (anstelle der Systemzeit) in einem zweiten, kombinierten Bag aufgezeichnet:

- `/camera/image_rect` – das entzerrte Bild,
- `/camera/camera_info` – die zugehörigen Kameraparameter,
- `/tagslam/tag_detections` – die Markerdetektionen,
- `/tf_static` – die statischen Koordinatentransformationen,
- `/clock` – die Zeit aus dem Eingangs-Bag, anstelle der Systemzeit.

Dieser kombinierte Bag enthält alle Eingangsdaten für die nachfolgende Kartenerstellung.

5.2.5 Kartenerstellung und Konfiguration von Tagslam

Die eigentliche Karte wird mit dem Knoten `tagslam_from_bag` aus dem Paket TagSLAM erstellt. Dieser liest den kombinierten Bag aus der vorigen Stufe ein und schätzt aus den enthaltenen Markerdetektionen die Lage aller Marker zueinander. Die Optimierung der Karte erfolgt nach dem in den Abschnitten 3.1.3.2 und 3.1.3.3 beschriebenen Prinzip. Der Knoten ist ausschließlich für ROS2 Jazzy implementiert, weshalb die Kartenerstellung unter dieser Version durchgeführt wird.

Der Knoten benötigt drei Konfigurationsdateien: `cameras.yaml` mit den Kamera-Intrinsics, `camera_poses.yaml` mit der Pose der Kamera sowie `tagslam.yaml` mit den Parametern der Optimierung und den Startwerten der Marker. Die Datei `cameras.yaml` entspricht der bereits in Abschnitt 5.2.3 beschriebenen Datei.

Die Datei `camera_poses.yaml` gibt über eine 6×6 -Kovarianzmatrix R an, wie starr die Kamera während der Kartierung gehalten wird. Die sechs Dimensionen entsprechen den drei translatorischen und drei rotatorischen Freiheitsgraden der Kamerapose. Da die Kamera bewegt wird und ihre Pose nicht vorab bekannt ist, sind Position und Rotation auf null gesetzt, während die Diagonalwerte der Matrix sehr groß gewählt sind. Ein großer Wert entspricht einer großen Unsicherheit, sodass die vorgegebene Nullpose die Optimierung nicht einschränkt und die Kamerapose allein aus den Markerbeobachtungen bestimmt wird.

Die Datei `tagslam.yaml` enthält neben den bereits in Abschnitt 5.2.3 beschriebenen Angaben (Tag-Identifikationsnummer und zugehörige Größe) die Parameter der Optimierung. Die wesentlichen Parameter sind in Tabelle 5.1 zusammengefasst.

Parameter	Bedeutung
<code>optimizer_mode</code>	Wahl zwischen schneller und genauerer Optimierung
<code>minimum_viewing_angle</code>	Mindestwinkel, unter dem ein Marker noch verwertet wird
<code>minimum_tag_area</code>	Mindestfläche eines Markers im Bild in Pixeln
<code>max_num_incremental_opt</code>	Anzahl der inkrementellen Optimierungsschritte
<code>pixel_noise</code>	Angenommenes Messrauschen der Eckpunkte in Pixeln
<code>max_subgraph_error</code>	Zulässiger Fehler, bis zu dem eine Teillösung akzeptiert wird
<code>use_approximate_sync</code>	Zeitliche Synchronisierung mit Toleranz

Tabelle 5.1: Wesentliche Parameter der Optimierung in `tagslam.yaml`

Für jeden Marker werden eine Startlage sowie das zugehörige Positions- und Rotationsrauschen (`position_noise`, `rotation_noise`) jeweils für alle drei Achsen angegeben. Das Rauschen bestimmt, wie stark ein Marker an seine vorgegebene Lage gebunden wird: Ein kleines Rauschen hält den Marker nahezu fest, ein großes Rauschen lässt seine Lage während der Optimierung frei. Über den Eintrag `fixed: true` kann ein Marker vollständig fixiert werden.

Auf diese Weise wird ein Marker als fester Ursprung definiert, der als globale Referenz dient, auf die alle übrigen Marker bezogen werden. In dieser Arbeit übernimmt Tag 0 diese Rolle. Grundsätzlich können auch mehrere Marker fixiert werden, etwa wenn deren Lage bereits genau bekannt ist.

Das Ergebnis ist eine konsistente Karte, die für jeden Marker dessen Identifikationsnummer, Größe, Lage und das verbleibende Rauschen enthält. Listing 5.4 zeigt einen Auszug am Beispiel zweier Marker.

Quellcode 5.4: Auszug einer erzeugten Karte

```
1 tags:
2   - id: 0
3     size: 0.16700000
4     pose:
5       position: {x: 0.0, y: 0.0, z: 0.0}
6       rotation: {x: 0.0, y: 0.0, z: 0.0}
7       position_noise: {x: 1.0e-06, y: 1.0e-06, z: 1.0e-06}
8       rotation_noise: {x: 1.0e-06, y: 1.0e-06, z: 1.0e-06}
9   - id: 1
10    size: 0.16700000
11    pose:
12      position: {x: 0.00499362, y: 0.60709535, z: -3.0e-08}
13      rotation: {x: 0.0, y: 0.0, z: 0.03398915}
14      position_noise: {x: 0.00012186, y: 0.00007138, z: 1.0e-06}
15      rotation_noise: {x: 1.0e-06, y: 1.0e-06, z: 0.00033940}
```

Die Position eines Markers ist in Metern relativ zum Ursprung angegeben, die Orientierung im Rodrigues-Format als Rotationsvektor (siehe Abschnitt 2.2.2). Am Beispiel von Tag 0 ist die Wirkung der Fixierung erkennbar: Seine Lage liegt exakt im Ursprung, und das zugehörige Rauschen ist sehr klein. Das zu jedem Marker ausgegebene Rauschen beschreibt die verbleibende Unsicherheit der geschätzten Lage und bildet die Referenz für die spätere Lokalisierung.

6 Implementierung der Positionierung

Dieses Kapitel beschreibt die im Flug ablaufende Positionsbestimmung des Fluggeräts. Im Gegensatz zur Kartierung (Kapitel 5) läuft die Verarbeitung hier zur Laufzeit auf dem Companion-Computer. Die Kette folgt dem in Abschnitt ?? dargestellten Datenfluss: Bildaufnahme, Entzerrung, Bildverarbeitung, Positionsbestimmung, Kommunikation mit der Flugsteuerung und schließlich die Flugsteuerung selbst. Die ersten drei Schritte entsprechen methodisch denen der Kartierung, werden aber mit höherer Bildrate und auf der GPU des Jetson ausgeführt. In nachfolgenden Abschnitt wird auf die konkrete Softwarearchitektur zur Positionierung eingegangen.

6.1 Softwarearchitektur der Positionierung

Die Positionierung ist auf zwei Ausführungsumgebungen aufgeteilt: den Host des Companion-Computers und einen Container. Diese Trennung ergibt sich aus der Verwendung von Isaac ROS für die hardwarebeschleunigten Schritte. Isaac ROS ist eine von NVIDIA bereitgestellte Sammlung von ROS2-Paketen, die für die Ausführung auf den GPUs der Jetson-Plattform optimiert sind. Die Pakete werden in einem vorkonfigurierten Container ausgeführt, der die benötigten Bibliotheken und Treiber bereitstellt. Innerhalb dieses Containers werden die Entzerrung und die Markererkennung auf der GPU ausgeführt, was gegenüber einer CPU-basierten Verarbeitung eine höhere Bildrate ermöglicht. Die übrigen Schritte laufen direkt auf dem Host des Jetson. Dort erfolgen die Bildaufnahme über gscam, die Positionsbestimmung aus den Markerdetektionen sowie die Kommunikation mit der Flugsteuerung. Container und Host kommunizieren über ROS2-Topics. Der Datenfluss zwischen den Knoten ist in Abbildung 6.1 dargestellt. Die Bildaufnahme stellt das Kamerabild auf `/camera_1/image_raw` mit den zugehörigen Parametern auf `/camera_1/camera_info` bereit. Im Container entzerrt der Knoten `RectifyNode` das Bild und gibt es als `/camera_1/image_rect` aus. Der Knoten `AprilTagNode` detektiert darauf die Marker und veröffentlicht die Detektionen auf `/camera_1/tag_detections`. Auf dem Host bestimmt der Knoten `vision_pose_uvrce_node` aus diesen Detektionen die Pose des Fluggeräts und übergibt sie über `/fmu/in/vehicle_visual_odometry` an die Flugsteuerung.

Abbildung 6.1: Knoten und Topics der Positionierung, aufgeteilt nach Host und Container

6.2 Bildaufnahme Positionierung

Die Bildaufnahme erfolgt wie bei der Kartierung (Abschnitt 5.2.2) über einen gscam-Knoten. Auch hier wird wie bei der Implementierung der Kartierung exemplarisch die Logitech C922 verwendet. Wie in Abschnitt ?? beschrieben, muss die Kalibrierung für jede Kamera und für die hier verwendete Auflösung von 1280×720 Pixeln durchgeführt werden.

Listing 6.1 zeigt die geänderte Konfiguration.

Quellcode 6.1: Parameter des gscam-Knotens zur Bildaufnahme im Flug

```

1 'gscam_config': (
2     'v4l2src device=/dev/video0 do-timestamp=true ! '
3     'image/jpeg,width=1280,height=720,framerate=60/1 ! '
4     'queue leaky=downstream max-size-buffers=1 ! '
5     'nvv4l2decoder mjpeg=1 ! '
6     'queue leaky=downstream max-size-buffers=1 ! '
7     'nvvidconv ! '
8     'video/x-raw,format=BGRx ! '
9     'videoconvert ! '
10    'video/x-raw,format=RGB,framerate=60/1'
11 ),
12 'camera_name': 'camera_1',
13 'frame_id': 'camera_1_optical_frame',
14 'camera_info_url': camera_info_url,
15 'use_gst_timestamps': True,
16 'sync_sink': False,
17 'preroll': False,
18 'image_encoding': 'rgb8',
19 'use_sensor_data_qos': False,

```

Die beiden wesentlichen Unterschiede betreffen die Bildrate und die Dekodierung. Die Kamera wird hier mit einer Auflösung von 1280×720 Pixeln bei 60 fps betrieben. Die deutlich höhere Bildrate ist notwendig, da die Positionsbestimmung im Flug fortlaufend erfolgt und eine geringe Latenz erfordert. Die Auflösung von 1280×720 Pixeln wurde gewählt, da die Logitech C922 nur bei dieser Auflösung 60 fps liefert. Möglich wird die hohe Bildrate durch die hardwarebeschleunigte Dekodierung des MJPEG-Streams über `nvv4l2decoder` und `nvvidconv` auf der GPU des Jetson. Ohne die hardwarebeschleunigte Dekodierung sinkt die Bildrate über die Zeit, da die CPU mit der Dekodierung nicht hinterherkommt und ein Puffer in der Pipeline vollläuft.

6.2.1 Kamerakalibrierung Positionierung

Schreibe mir den Abschnitt zur Kalibrierung. sag dass für die neue Auflösung neue Parameter nach dem referenzierten Abschnitt ermittelt werden müssen.

6.2.2 Kamerakalibrierung Positionierung

Die intrinsische Kalibrierung der Flugkamera folgt demselben Lochkameramodell und Verzeichnungsmodell wie in Abschnitt ?? beschrieben. Die Kalibrierung wird für das im Flug verwendete 1280×720 -Profil separat durchgeführt, da Brennweite und Hauptpunkt von der Auflösung abhängen. Das Ergebnis wird als `camera_info` bereitgestellt und in der nachfolgenden Entzerrung verwendet.

6.3 ab hier ist vorsicht geboten

6.4 Entzerrung

Die Entzerrung erfolgt im Flug über den Knoten `RectifyNode` aus dem Paket `isaac_ros_image_proc`. Dieser Knoten läuft GPU-beschleunigt innerhalb eines `Composable-Node-Containers` und entfernt die Verzerrung anhand der Kalibrierparameter. Das entzerrte Bild wird als `/camera_1/image_rect` mit den zugehörigen Parametern `/camera_1/camera_info_rect` ausgegeben. Wie in der Kartierung gilt: Nach der Entzerrung sind die Verzeichnungskoeffizienten null, und es werden die Parameter der Projektionsmatrix verwendet.

6.5 Bildverarbeitung

Die Markererkennung übernimmt der Knoten `AprilTagNode` aus dem Paket `isaac_ros_apriltag`. Er detektiert im entzerrten Bild Marker der Familie `tag36h11` mit einer konfigurierten Größe von 0,167m und maximal 40 Markern pro Bild. Die Detektion läuft GPU-beschleunigt und ist über das NITROS-Framework speichereffizient an die Entzerrung gekoppelt. Für jeden erkannten Marker liefert der Knoten dessen Identifikationsnummer sowie die relative Pose zwischen Kamera und Marker als Topic `/camera_1/tag_detections`. Diese Detektionen bilden den Eingang der Positionsbestimmung.

6.6 Positionsbestimmung

Die Positionsbestimmung übernimmt der Knoten `vision_pose_uvrce_node`. Er berechnet aus den Markerdetektionen die Pose des Fluggeräts in der Karte und übergibt sie an die Kommunikation. Der Ablauf gliedert sich in vier Schritte: die Posenberechnung je Marker, die Ausreißerfilterung, die gewichtete Fusion und die Umrechnung auf den Drohnenmittelpunkt.

6.6.1 Posenberechnung je Marker

Jede Detektion liefert die Transformation $\mathbf{T}_{\text{camera}}^{\text{tag}}$ zwischen Kamera und Marker. Da die Lage jedes Markers in der Karte aus der Kartierung bekannt ist ($\mathbf{T}_{\text{map}}^{\text{tag}}$), lässt sich daraus die Kamerapose in der Karte berechnen:

$$\mathbf{T}_{\text{map}}^{\text{camera}} = \mathbf{T}_{\text{map}}^{\text{tag}} \cdot (\mathbf{T}_{\text{camera}}^{\text{tag}})^{-1}. \quad (6.1)$$

Jeder sichtbare Marker erzeugt somit eine eigene Hypothese für die Kamerapose. Jeder Hypothese wird ein Gewicht zugewiesen, das mit dem Quadrat der Markerdistanz d abnimmt:

$$w = \frac{1}{d^2}. \quad (6.2)$$

Nahe Marker werden dadurch stärker gewichtet, da ihre Posenschätzung genauer ist.

6.6.2 Ausreißerfilterung

Bei mehreren sichtbaren Markern müssen die Hypothesen aus Gleichung (6.1) übereinstimmen. Fehldetektionen oder mehrdeutige Markerposen erzeugen jedoch abweichende Hypothesen. Diese werden über ein RANSAC-Verfahren entfernt, das das größte Set konsistenter Hypothesen (Inlier) sucht. Da das Rauschen der Posenschätzung mit der Distanz quadratisch zunimmt, wird die Inlier-Schwelle τ distanzabhängig skaliert:

$$\tau = \tau_{\text{base}} \cdot \left(\frac{d_{\text{mean}}}{d_{\text{ref}}} \right)^2, \quad (6.3)$$

mit der Basisschwelle $\tau_{\text{base}} = 0,015$ m bei der Referenzdistanz $d_{\text{ref}} = 0,30$ m. Ein Konsens gilt erst ab einem Inlier-Anteil von 70 % als zuverlässig. Bei wenigen Markern wird das Set vollständig durchsucht, was die Filterung über aufeinanderfolgende Bilder deterministisch macht.

6.6.3 Gewichtete Fusion

Die verbleibenden Inlier-Hypothesen werden zu einer Kamerapose zusammengefasst. Die Position ergibt sich als gewichtetes Mittel über alle Inlier i :

$$\mathbf{t}_{\text{map}}^{\text{camera}} = \frac{\sum_i w_i \mathbf{t}_i}{\sum_i w_i}. \quad (6.4)$$

Die Orientierung wird analog über die zugehörigen Quaternionen gemittelt.

6.6.4 Umrechnung auf den Drohnenmittelpunkt

Die Kamera ist starr, aber außerhalb des Schwerpunkts am Fluggerät montiert. Die für die Flugsteuerung relevante Pose ist jedoch die des Drohnenmittelpunkts. Mit der statischen Transformation $\mathbf{T}_{\text{camera}}^{\text{drone}}$ (siehe Abschnitt 6.6.6) folgt:

$$\mathbf{T}_{\text{map}}^{\text{drone}} = \mathbf{T}_{\text{map}}^{\text{camera}} \cdot \mathbf{T}_{\text{camera}}^{\text{drone}}. \quad (6.5)$$

Das Ergebnis ist die Pose des Fluggeräts im Kartenkoordinatensystem.

6.6.5 Kovarianzschätzung

Zusätzlich wird zu jeder Pose eine Unsicherheit geschätzt, die die Flugsteuerung zur Gewichtung der Messung nutzt. Die Standardabweichung pro Marker hängt von dessen Distanz d und Größe s ab:

$$\sigma = \sigma_{\text{base}} \cdot \left(\frac{d}{d_{\text{ref}}} \right)^2 \cdot \frac{s_{\text{ref}}}{s}, \quad (6.6)$$

mit den Basiswerten $\sigma_{\text{base,pos}} = 0,012$ m und $\sigma_{\text{base,rot}} = 0,0122$ rad bei $d_{\text{ref}} = 1,5$ m und $s_{\text{ref}} = 0,17$ m. Entfernte und kleine Marker erhalten so eine größere Unsicherheit. Liegen mehrere Marker vor, werden ihre Varianzen über eine inverse-Varianz-Gewichtung zu einer Gesamtkovarianz kombiniert.

6.6.6 Extrinsische Kamerakalibrierung

Die in Gleichung (6.5) benötigte Transformation $\mathbf{T}_{\text{camera}}^{\text{drone}}$ beschreibt die feste Montage der Kamera relativ zum Drohnenmittelpunkt. Sie wird vorab über den Knoten `drone_camera_calibration_node` bestimmt.

Grundlage ist ein spezieller Marker, der den Drohnenmittelpunkt definiert und dessen Lage in der Karte bekannt ist ($\mathbf{T}_{\text{map}}^{\text{drone}}$). Während der Kalibrierung wird das Fluggerät so positioniert, dass sowohl dieser Marker als auch die Kartenmarker sichtbar sind. Aus der laufend bestimmten Kamerapose $\mathbf{T}_{\text{map}}^{\text{camera}}$ ergibt sich die gesuchte Transformation zu:

$$\mathbf{T}_{\text{drone}}^{\text{camera}} = \left(\mathbf{T}_{\text{map}}^{\text{drone}} \right)^{-1} \cdot \mathbf{T}_{\text{map}}^{\text{camera}}. \quad (6.7)$$

Da eine Einzelmessung verrauscht ist, werden 100 Messungen aufgenommen und gemittelt. Die Position wird arithmetisch gemittelt, die Orientierung über eine vorzeichenkorrigierte Quaternionenmitteilung. Die Vorzeichenkorrektur ist notwendig, da $\mathbf{q}$ und $-\mathbf{q}$ dieselbe Rotation beschreiben und sich bei naiver Mittelung gegenseitig auslöschen könnten. Das Ergebnis wird als statische Transformation gespeichert und im Flug verwendet.

6.7 Kommunikation mit der Flugsteuerung

Die Kommunikation überführt die berechnete Pose in das Format der Flugsteuerung und übergibt sie an PX4. Die Anbindung erfolgt über uXRCE-DDS, das die Pose direkt als `VehicleOdometry`-Nachricht auf dem Topic `/fmu/in/vehicle_visual_odometry` bereitstellt.

6.7.1 Koordinatentransformation

Die interne Verarbeitung erfolgt im ROS-üblichen ENU/FLU-System, PX4 erwartet die Daten jedoch im NED/FRD-System. Die Position wird daher umgerechnet:

$$(x, y, z)_{\text{ENU}} \rightarrow (y, x, -z)_{\text{NED}}. \quad (6.8)$$

Die Orientierung wird über zwei feste Hilfsrotationen umgerechnet, eine für den Welt- und eine für den Körperframe:

$$\mathbf{q}_{\text{NED/FRD}} = \mathbf{q}_{\text{ENU} \rightarrow \text{NED}} \otimes \mathbf{q}_{\text{ENU/FLU}} \otimes \mathbf{q}_{\text{FLU} \rightarrow \text{FRD}}. \quad (6.9)$$

Anschließend wird die Quaternionenreihenfolge von der ROS-Konvention (x, y, z, w) auf die PX4-Konvention (w, x, y, z) umgestellt.

6.7.2 Zeitbezug und Ausgabe

Die Nachricht wird mit einem Zeitstempel versehen, der mit der Zeitbasis von PX4 übereinstimmt. Da der Companion-Computer und PX4 auf einer gemeinsamen Zeitbasis laufen, wird der lokale Zeitstempel direkt verwendet. Das Feld `pose_frame` wird auf NED gesetzt, und die zuvor geschätzte

Kovarianz wird in die Varianzfelder der Nachricht übernommen. Die Pose wird mit einer Rate von 60 Hz publiziert.

6.8 Flugsteuerung

Die Flugsteuerung läuft auf dem PX4-Flugcontroller und regelt das Fluggerät. Die eingespeiste Pose wird im Zustandsschätzer EKF2 mit den Daten der IMU fusioniert. Dadurch bleibt die Zustandsschätzung auch dann stabil, wenn kurzzeitig keine Marker sichtbar sind und somit keine neue Pose eintrifft. Die so geschätzte Pose bildet die Grundlage für die Lage- und Positionsregelung des Fluggeräts.

7 Systemintegration

8 Evaluation des Greifens

8.1 Erfolgskriterien - Erfolgsquote und Fehlerrate

8.2 Erkennungsgenauigkeit

8.3 Positioniergenauigkeit

8.4 Zeitliche Performance und Echtzeitfähigkeit

9 Zusammenfassung und Ausblick

Literatur

- [1] H. Studiawan und K.-K. R. Choo, *Drone and UAV Forensics: A Hands-On Approach*. Springer Nature Switzerland, 2026, ISBN: 9783031935114. DOI: 10.1007/978-3-031-93511-4.
- [2] Y.-T. Wu, Z. Qin, A. Eizad und S.-K. Lyu, "Numerical investigation of the mechanical component design of a hexacopter drone for real-time fine dust monitoring", *Journal of Mechanical Science and Technology*, Jg. 35, Nr. 7, S. 3101–3111, 2021, ISSN: 1976-3824. DOI: 10.1007/s12206-021-0632-y.
- [3] P. Modellbau, *Holybro X650*, <https://www.premium-modellbau.de/holybro-holybro-x650-development-kit-pixhawk-6c-m10-gps-433-mhz-30180>, Zugriff am 30.05.2026, 2026.
- [4] ISO, "Road Vehicles – Vehicle Dynamics and Road-Holding Ability – Vocabulary", International Organization for Standardization, Geneva, Switzerland, Standard ISO 8855:2011, 2011.
- [5] ISO, "Robots and Robotic Devices – Coordinate Systems and Motion Nomenclatures", International Organization for Standardization, Geneva, Switzerland, Standard ISO 9787:2013, Mai 2013.
- [6] PX4 Development Team, *Using Vision or Motion Capture Systems for Position Estimation*, https://docs.px4.io/main/en/ros/external_position_estimation, Zugriff am 31.05.2026, 2024.
- [7] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer International Publishing, 2022, ISBN: 9783030343729. DOI: 10.1007/978-3-030-34372-9.
- [8] B. Siciliano, L. Sciavicco, L. Villani und G. Oriolo, *Robotics*. Springer London, 2009, ISBN: 9781846286421. DOI: 10.1007/978-1-84628-642-1.
- [9] *Springer Handbook of Robotics*. Springer International Publishing, 2016, ISBN: 9783319325521. DOI: 10.1007/978-3-319-32552-1.
- [10] Q. Quan, *Introduction to Multicopter Design and Control*. Springer Singapore, 2017, ISBN: 9789811033827. DOI: 10.1007/978-981-10-3382-7.
- [11] G. Bradski und A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008, ISBN: 978-0596516130. Adresse: <https://learning.oreilly.com/library/view/learning-opencv/9780596516130/ch11s05.html>.
- [12] J. S. Dai, "Euler–Rodrigues formula variations, quaternion conjugation and intrinsic connections", *Mechanism and Machine Theory*, Jg. 92, S. 144–152, 2015, ISSN: 0094-114X. DOI: <https://doi.org/10.1016/j.mechmachtheory.2015.03.004>. Adresse: <https://www.sciencedirect.com/science/article/pii/S0094114X15000415>.
- [13] V. Capuano, L. Xu und J. Estrada Benavides, "Smartphone MEMS Accelerometer and Gyroscope Measurement Errors: Laboratory Testing and Analysis of the Effects on Positioning Performance", *Sensors*, Jg. 23, Nr. 17, S. 7609, 2023, ISSN: 1424-8220. DOI: 10.3390/s23177609.

- [14] C. Chen und X. Pan, "Deep Learning for Inertial Positioning: A Survey", *IEEE Transactions on Intelligent Transportation Systems*, 2024. DOI: 10.1109/TITS.2024.3381161. arXiv: 2303.03757 [cs.RO].
- [15] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems", *Transactions of the ASME – Journal of Basic Engineering*, Jg. 82, Nr. 1, S. 35–45, 1960. DOI: 10.1115/1.3662552.
- [16] Y. Kim und H. Bang, "Introduction to Kalman Filter and Its Applications", in *Introduction and Implementations of the Kalman Filter*, F. Govaers, Hrsg., London: IntechOpen, 2018, Kap. 2. DOI: 10.5772/intechopen.80600. Adresse: <https://doi.org/10.5772/intechopen.80600>.
- [17] J. A. Mendoza-Mendoza, V. J. Gonzalez-Villela, G. Sepulveda-Cervantes, M. Mendez-Martinez und H. Sossa-Azuela, *Advanced Robotic Vehicles Programming: An Ardupilot and Pixhawk Approach*. Apress, 2020, ISBN: 9781484255315. DOI: 10.1007/978-1-4842-5531-5.
- [18] PX4 Development Team, *Holybro Pixhawk 6X*, https://docs.px4.io/v1.16/en/flight_controller/pixhawk6x, Abgerufen am 04.06.2026.
- [19] N. Y. Ko, W. Youn, I. H. Choi, G. Song und T. S. Kim, "Features of Invariant Extended Kalman Filter Applied to Unmanned Aerial Vehicle Navigation", *Sensors*, Jg. 18, Nr. 9, S. 2855, Aug. 2018, ISSN: 1424-8220. DOI: 10.3390/s18092855.
- [20] PX4 Development Team, *PX4 Architectural Overview*, <https://docs.px4.io/main/en/concept/architecture.html>, Abgerufen am 07.06.2026, 2025.
- [21] MAVLink Development Team, *MAVLink Developer Guide*, <https://mavlink.io/en/>, Erstveröffentlichung 2009 durch Lorenz Meier; abgerufen am 07.06.2026, 2009.
- [22] PX4 Development Team, *MAVLink Messaging – PX4 Autopilot Documentation*, <https://docs.px4.io/main/en/middleware/mavlink.html>, Abgerufen am 07.06.2026, 2025.
- [23] PX4 Development Team, *ULog File Format – PX4 Autopilot Documentation*, https://docs.px4.io/main/en/dev_log/ulog_file_format.html, Abgerufen am 07.06.2026, 2025.
- [24] PX4 Development Team, *Flight Reporting – PX4 Autopilot Documentation*, https://docs.px4.io/main/en/getting_started/flight_reporting.html, Abgerufen am 07.06.2026, 2025.
- [25] M. A. Fischler und R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography", in *Readings in Computer Vision*. Elsevier, 1987, S. 726–740, ISBN: 9780080515816. DOI: 10.1016/b978-0-08-051581-6.50070-2.
- [26] Open Robotics, *ROS 2 Middleware Implementations*, <https://docs.ros.org/en/humble/Concepts/Advanced/About-Middleware-Implementations.html>, ROS 2 Humble Hawksbill; abgerufen am 07.06.2026, 2022.

- [27] Open Robotics, *Nodes – ROS 2 Documentation: Humble*, <https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html>, ROS 2 Humble Hawksbill; abgerufen am 07.06.2026, 2022.
- [28] Open Robotics, *Topics – ROS 2 Documentation: Humble*, <https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html>, ROS 2 Humble Hawksbill; abgerufen am 07.06.2026, 2022.
- [29] Open Robotics, *Recording and Playing Back Data – ROS 2 Documentation: Humble*, <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>, ROS 2 Humble Hawksbill; abgerufen am 07.06.2026, 2022.
- [30] I. Jarraya, A. Al-Batati, M. B. Kadri u. a., “Gnss-denied unmanned aerial vehicle navigation: analyzing computational complexity, sensor fusion, and localization methodologies”, *Satellite Navigation*, Jg. 6, Nr. 1, Apr. 2025, ISSN: 2662-1363. DOI: 10.1186/s43020-025-00162-z.
- [31] H. Zhou, Y. Zhao, X. Xiong, Y. Lou und S. Kamal, “IMU Dead-Reckoning Localization with RNN-IEKF Algorithm”, in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, S. 11 382–11 387. DOI: 10.1109/IROS47612.2022.9982087.
- [32] J. T. Jang, A. Santamaria-Navarro, B. T. Lopez und A.-a. Agha-mohammadi, “Analysis of State Estimation Drift on a MAV Using PX4 Autopilot and MEMS IMU During Dead-reckoning”, in *2020 IEEE Aerospace Conference*, 2020, S. 1–11. DOI: 10.1109/AERO47225.2020.9172736.
- [33] J. Bednář, M. Petrлік, K. C. T. Vivaldini und M. Saska, “Deployment of Reliable Visual Inertial Odometry Approaches for Unmanned Aerial Vehicles in Real-world Environment”, in *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2022, S. 167–176. DOI: 10.1109/ICUAS54217.2022.9836067.
- [34] A. Khole, A. Thakar, S. Shende und V. Karajkhede, “A Comprehensive Study on Simultaneous Localization and Mapping (SLAM): Types, Challenges and Applications”, in *2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, 2023, S. 643–650. DOI: 10.1109/ICSCSS57650.2023.10169695.
- [35] D. Cai, R. Li, Z. Hu, J. Lu, S. Li und Y. Zhao, “A comprehensive overview of core modules in visual SLAM framework”, *Neurocomputing*, Jg. 590, S. 127 760, 2024, ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2024.127760>. Adresse: <https://www.sciencedirect.com/science/article/pii/S0925231224005319>.
- [36] S. Gong, *Loop Closure*, <https://stevengong.co/notes/Loop-Closure>, Abgerufen am 06.06.2026.
- [37] T. Lai, *A Review on Visual-SLAM: Advancements from Geometric Modelling to Learning-based Semantic Scene Understanding*, 2022. DOI: 10.48550/ARXIV.2209.05222.
- [38] F. Dellaert und M. Kaess, “Factor Graphs for Robot Perception”, *Foundations and Trends in Robotics*, Jg. 6, Nr. 1-2, S. 1–139, 2017, ISSN: 1935-8261. DOI: 10.1561/23000000043.

- [39] D. Cai, R. Li, Z. Hu, J. Lu, S. Li und Y. Zhao, "A comprehensive overview of core modules in visual SLAM framework", *Neurocomputing*, Jg. 590, S. 127 760, 2024, ISSN: 0925-2312. DOI: 10.1016/j.neucom.2024.127760.
- [40] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. Leonard und F. Dellaert, "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree", *International Journal of Robotic Research - IJRR*, Jg. 31, S. 216–235, Mai 2012. DOI: 10.1177/0278364911430419.
- [41] F. Dellaert, "Factor Graphs and GTSAM: A Hands-on Introduction", Georgia Institute of Technology, Techn. Ber., 2012. Adresse: <https://gtsam.org/tutorials/intro.html>.
- [42] B. Pfrommer und K. Daniilidis, *TagSLAM: Robust SLAM with Fiducial Markers*, 2019. arXiv: 1910.00679 [cs.RO].
- [43] X. Yue, Y. Zhang und M. He, *LiDAR-based SLAM for robotic mapping: state of the art and new frontiers*, 2023. DOI: 10.48550/ARXIV.2311.00276.
- [44] M. Kalaitzakis, B. Cain, S. Carroll, A. Ambrosi, C. Whitehead und N. Vitzilaios, "Fiducial Markers for Pose Estimation: Overview, Applications and Experimental Comparison of the ARTag, AprilTag, ArUco and STag Markers", *Journal of Intelligent and Robotic Systems*, Jg. 101, Nr. 4, März 2021, ISSN: 1573-0409. DOI: 10.1007/s10846-020-01307-9.
- [45] M. Loipfuehrer, "Efficient and Robust Circle Grids for Fiducial Detection", Supervisor: Prof. Dr. Daniel Cremers, Advisor: Nikolaus Demmel, Master's Thesis, Technische Universität München, Mai 2022. Adresse: https://cvg.cit.tum.de/_media/members/demmeln/loipfuehrer2022ma.pdf.
- [46] F. Siri, J. Song und M. Svinin, "Experimental verification of coverage control of multi-agent systems with obstacle avoidance", *The Journal of Engineering*, Jg. 2024, Dez. 2024. DOI: 10.1049/tje2.70045.
- [47] M. Fiala, "ARTag, a fiducial marker system using digital techniques", in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 05)*, Bd. 2, 2005, S. 590–596. DOI: 10.1109/CVPR.2005.74.
- [48] E. Olson, "AprilTag: A robust and flexible visual fiducial system", in *2011 IEEE International Conference on Robotics and Automation*, 2011, S. 3400–3407. DOI: 10.1109/ICRA.2011.5979561.
- [49] J. Wang und E. Olson, "AprilTag 2: Efficient and robust fiducial detection", in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, S. 4193–4198. DOI: 10.1109/IROS.2016.7759617.
- [50] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas und M. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion", *Pattern Recognition*, Jg. 47, Nr. 6, S. 2280–2292, 2014, ISSN: 0031-3203. DOI: <https://doi.org/10.1016/j.patcog.2014.01.005>. Adresse: <https://www.sciencedirect.com/science/article/pii/S0031320314000235>.

- [51] B. Benligiray, C. Topal und C. Akinlar, "STag: A stable fiducial marker system", *Image and Vision Computing*, Jg. 89, S. 158–169, 2019, ISSN: 0262-8856. DOI: 10.1016/j.imavis.2019.06.007.
- [52] Seeed Studio, *reComputer Mini J3011 – Intelligent Edge AI Computer with NVIDIA® Jetson™ Orin™ Nano 8GB*, 2024. Adresse: <https://www.seeedstudio.com/reComputer-Mini-J3011-p-6356.html> (besucht am 09.06.2026).
- [53] PICHLER GmbH. "LiPo Akku RED POWER XT 5400 – 11,1V". Abgerufen am 14. Juni 2026. (), Adresse: <https://pichler.de/LiPo-Akku-RED-POWER-XT-5400-111V> (besucht am 14.06.2026).
- [54] Holybro. "PM02 (V3) Power Module". Abgerufen am 14. Juni 2026, PX4 User Guide. (), Adresse: https://docs.px4.io/main/en/power_module/holybro_pm02.html (besucht am 14.06.2026).
- [55] Technische Hochschule Nürnberg Georg Simon Ohm. "AerodrOhm – Flugroboterhalle". Abgerufen am 12. Juni 2026. (2025), Adresse: <https://www.th-nuernberg.de/forschung-innovation/forschungseinrichtungen/ohm-innovation-center/forschung-am-oic/aerodrohm/> (besucht am 12.06.2026).
- [56] J. Tang, Y. Chen, X. Niu u. a., "LiDAR Scan Matching Aided Inertial Navigation System in GNSS-Denied Environments", *Sensors*, Jg. 15, Nr. 7, S. 16 710–16 728, 2015, ISSN: 1424-8220. DOI: 10.3390/s150716710.
- [57] Q. Peng, X. Zhao, R. Dang und Z. Xiang, "RWT-SLAM: Robust Visual SLAM for Weakly Textured Environments", in *2024 IEEE Intelligent Vehicles Symposium (IV)*, 2024, S. 913–919. DOI: 10.1109/IV55156.2024.10588822.
- [58] Livox Technology. "Livox Mid-360S". Abgerufen am 13. Juni 2026. (2025), Adresse: <https://www.livoxtech.com/de/mid-360s> (besucht am 13.06.2026).
- [59] Logitech. "C922 Pro Stream Webcam". Abgerufen am 13. Juni 2026. (), Adresse: <https://www.logitech.com/de-de/shop/p/c922-pro-stream-webcam> (besucht am 13.06.2026).
- [60] Stereolabs. "ZED Mini Stereo Camera". Abgerufen am 13. Juni 2026. (), Adresse: <https://www.stereolabs.com/en-de/store/products/zed-mini> (besucht am 13.06.2026).
- [61] Roboception GmbH. "TagDetect – rc_visard Documentation". Abgerufen am 13. Juni 2026. (), Adresse: <https://doc.rc-visard.com/latest/en/tagdetect.html> (besucht am 13.06.2026).
- [62] Z. Zhang, "A flexible new technique for camera calibration", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 22, Nr. 11, S. 1330–1334, 2000. DOI: 10.1109/34.888718.

Abbildungsverzeichnis

2.1	Vergleich der Koordinatensysteme [3]	3
2.2	Roll, Pitch und Yaw am Beispiel Holybro X650 [3]	4
2.3	Komponenten des Pixhawk 6X: FMU-Modul und Baseboard V2A [3]	10
3.1	Veranschaulichung des Loop-Closure-Effekts: (a) Reale Trajektorie ohne akkumulierten Fehler. (b) Geschätzte Trajektorie mit akkumuliertem Drift ohne Loop Closure. (c) Korrigierte Trajektorie nach Anwendung von Loop Closure mit reduziertem akkumuliertem Drift. vgl. [36]	17
3.2	Vereinfachter Faktorgraph eines SLAM-Problems, vgl. [37], [38], [39]	18
3.3	Übersicht gängiger Fiducial-Marker: a) ARTag; b) AprilTag; c) Aruco d) Stag [45], [46]	19
4.1	Hardware-Blockschaltbild mit Anschlussverdrahtung	25
4.2	Softwarearchitektur mit den Strängen zur Lokalisierung und zur Kartierung	25
4.3	Flughalle AerodrOhm am OIC der Technischen Hochschule Nürnberg [55]	27
5.1	Paarweiser Vergleich Bewertungskriterien der SLAM-Varianten	29
5.2	Nutzwertanalyse der SLAM-Varianten	31
6.1	Knoten und Topics der Positionierung, aufgeteilt nach Host und Container	39

Tabellenverzeichnis

2.1	Sensorbestückung des Pixhawk 6X [18]	10
4.1	Übersicht der eingesetzten Hardwarekomponenten	24
5.1	Wesentliche Parameter der Optimierung in <code>tagslam.yaml</code>	37

Quellcodeverzeichnis

5.1	Start des gscam-Knotens für die Kalibrierung	33
5.2	Start des Kalibrierknotens	33
5.3	Parameter des gscam-Knotens zur Bildaufnahme	35
5.4	Auszug einer erzeugten Karte	38
6.1	Parameter des gscam-Knotens zur Bildaufnahme im Flug	39

10 Anhang

Der Anhang enthält ergänzende Informationen, die für das Verständnis der Arbeit relevant sind, aber nicht direkt in den Haupttext integriert werden konnten. Dazu gehören beispielsweise ausführliche Tabellen, zusätzliche Abbildungen oder Quellcode, der für die Implementierung der beschriebenen Methoden verwendet wurde.